

配置中心Lion

正文 2.7 万字 (28.4 页) | 浏览 25 次 (1 人) | 全部 >

AI 速览 Lion是美团研发的分布式配置中心，核心能力包括实时配置推送、多环境管理、灰度发布和最终一致性保障。其核心价值在于通过中

简介

Lion是一个配置管理平台，可以实时推送配置的变更，对标行业中的nacos、Apoll等

Lion是一个面向分布式系统的配置管理平台，采用中心化数据管理，采用客户端/服务端的框架实现配置的统一管理和实时推送

特点

- 支持从多种数据源加载配置。比如环境变量、lion服务端配置、本地配置文件、zookeeper、http服务器和保存本地磁盘的当前工作配置。
- 支持多种配置获取。主要包括：项目配置、地区配置、泳道配置、公共组件配置、分组配置
- 支持多种环境的配置管理。开发、测试、生产
- 提供配置变更通知服务。配置的变更会实时通知到相关的客户端。配置变更的通知机制有监听器机制（全局、应用级、配置项）、异步通知支持
- 客户端会定时上报当前配置的值，这样在Lion的管理端就可以查询到某个客户端当前使用的配置的值。同时配置上报可以方便对配置进行治理。如一些很久都没有使用的配置，可以在Lion的管理端下线。ps不是很理解
- Lion客户端集成了Cat监控，可以在Cat上看到获取配置占用的时间，配置变更到达的延迟以及配置同步占用的时间。
- Lion客户端使用slf4j作为默认的日志组件

可以和下面合并

- **统一管理不同环境、不同Set泳道的配置**
 - 提供统一界面，集中式管理应用以及应用下不同环境（environment）、不同Set泳道的配置。
 - 支持从多种数据源加载配置。
- **配置变更实时生效**：用户发布配置后，客户端能秒级接收到最新的配置。



• **版本发布管理**：所有的配置发布都有版本概念，从而可以方便地支持配置的回滚。

- **具备灰度能力**：支持配置的灰度发布，只对部分客户端灰度生效，等观察一段时间没问题后再推给所有应用实例
- **权限管理、发布审核、操作审计**
 - 应用和配置的管理都有完善的权限管理、审核机制，减少人为修改的错误。
 - 所有的变更都有操作日志，方便追踪问题。
- **提供Java和C++原生客户端、OpenAPI**：方便应用集成。

作用

- 实现配置与代码分离、配置变更推送、历史版本管理、灰度发布等功能
- 中心化数据管理，实现秒级在线配置管理，并且保证所有节点的最终一致性

补充

Lion的出现将改变并解决这些问题。

1. Lion 是一个面向分布式系统的配置中心，采用中心化数据管理，通过客户端/服务器架构实现配置的统一管理和实时推送。
2. 通过使用Lion可实现配置与代码分离、配置变更推送、历史版本管理、灰度发布等功能，通过集中化的在线配置管理，应用可以实现秒级配置变更，且保证所有节点配置的最终一致性，工程师不用担心由于人为失误或缺少检查导致的部分节点表现异常，从而影响服务稳定性的情况。
3. Lion提供的操作日志、灰度发布、配置审核、变更检测等高级功能，便于工程师更为精细和安全的维护应用的状态。同时Lion提供的配置全链路监控将极大的改善配置变更推送的可观测性，方便用户查看跟踪配置使用情况。
4. Lion提供配置治理能力，通过配置全生命周期的使用数据收集和展示，方便业务方掌握配置使用情况，同时对一些长期不使用或者异常使用的配置进行治理。

一致性说明

Lion保证最终一致性

1. 所有客户端角度：最终一致，一定时间后（5s内）所有客户端读到的值最终达到完全一致。
2. 单个客户端角度：单调读一致，当客户端读取到某处变更后的值后，那么就不会再读到比这个值更旧的值。

Lion 使用过程需要注意什么

- 一、一个服务+环境的配置项 (key+value+comment) 的总和 $\leq 5M$ ，大于5M会写失败
- 二、Lion变更配置推送是秒级延迟，客户端缓存了配置值，在变更通知到达之前，缓存的配置值不会更改；
- 三、Lion保证最终一致性，同一个配置5秒内多次更改只保证最后一次更改值被通知到，不能捕捉每一次的更改

读写操作QPS

写：value大小32B，不同配置写：2200QPS；相同配置写：200QPS。注意：当前数据是Lion整体的写操作QPS，目前限流1000QPS，这个阈值是所有业务方共用。

读：

- HTTP读请求，不同配置：6500QPS；相同配置：7500QPS。
- 客户端读请求，500并发-5000QPS，150wQPS。

100%灰度和全量的区别

- Lion灰度百分比是以灰度发布创建时的客户端节点数量作为统计集合，100%灰度并不包括在这之后扩容的新客户端节点。
- 全量包括所有请求节点的客户端，包括新扩容的节点。

客户端请求与HTTP请求区别

- SDK请求，指访问配置服务集群(ConfigServer)获取配置，客户端缓存配置数据，读性能高。可参考：[Lion SDK接入文档](#)。
- HTTP请求，指访问API服务集群(APIServer)获取配置，读性能低。可参考：[Lion HTTP接口接入指南](#)，Java客户端也有封装HTTP接口：[动态配置HTTP接口](#)。

使用限制

从一致性和容量两个方面说起

1. 保证最终一致，单key更改QPS ≤ 1 ，更改频次高的情况下只保证最后一次更新能推送到客户端。
2. 容量限制，超过容量的操作将失败

a. 单key容量大小限制：500K

b. appkey下单环境容量限制：5M

适合：多节点配置的分发、应用对应用的配置依赖、应用业务场景动态推送等。

不适合：

1. 单个配置频繁变更（更改的QPS>1）
2. 一致性要求高于Lion的最终一致性（强一致性、顺序一致、因果一致等）ps后面两个一致性是什么？
3. 配置变更后立刻读的场景（写与读的间隔小于5s），Lion配置变更需要服务器推送到客户端

使用

管理端接入

- 1、需要有对应的Lion项目，需要根据对应的appkey来申请Lion项目；如果没有appkey需要先申请

客户端接入

1、引入pom依赖

方法一：inf-bom引入

```
1 <!-- inf-bom引入(推荐) -->
2 <dependencyManagement>
3   <dependencies>
4     <dependency>
5       <groupId>com.sankuai</groupId>
6       <artifactId>inf-bom</artifactId>
7       <!-- inf-bom 版本记录:https://km.sankuai.com/page/28122541 -->
8       <version>1.4.9</version>
9       <type>pom</type>
10      <scope>import</scope>
```

```
1      </dependency>
2  </dependencies>
3  </dependencyManagement>
4
5  <dependencies>
6      <dependency>
7          <groupId>com.dianping.lion</groupId>
8          <artifactId>lion-client</artifactId>
9      </dependency>
10 </dependencies>
```

方法二：指定版本引入

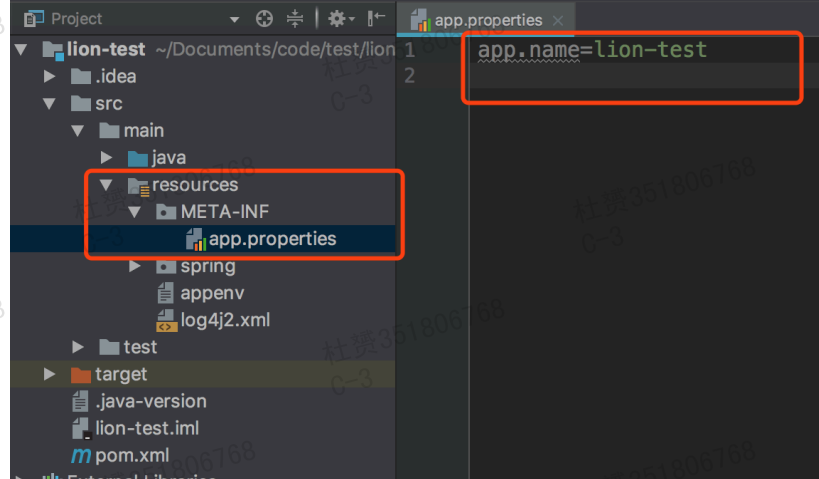
XML



```
1 <!-- 指定版本引入 -->
2 <dependencies>
3     <dependency>
4         <groupId>com.dianping.lion</groupId>
5         <artifactId>lion-client</artifactId>
6         <!-- lion-client 版本记录 https://km.sankuai.com/page/28353493 -->
7         <version>0.8.9.7</version>
8     </dependency>
9 </dependencies>
```

2、配置环境信息和指定appkey

在项目classpath路径下的 META-INF/app.properties 指定app.name=xxx（项目名），如下图所示



环境信息从以下appenv文件读取，appenv文件标准格式如下：

代码块

XML

- 1 env=环境定义字段
- 2 swimlane=泳道（可选），一般为空
- 3 cell=SET名（可选），一般为空

按照优先级从高到低分别为：

1. 通过System.setProperty("appenv.dir", \${path})指定appenv文件，参考：[基础组件支持动态环境配置用法说明](#)。
2. 从本地/data/webapps/appenv配置文件获取。如果是Windows 环境，根据程序运行目录的不同，appenv 文件需要放在 C:/data/webapps/appenv或D:/data/webapps/appenv或E:/data/webapps/appenv。
3. \$WAR_ROOT/appenv 兼容老版本，读取该文件
4. 如果这个文件也不存在，则默认 dev 环境

3.1、动态配置API使用

有两种方式**ConfigRepository**和**Lion**，两者都是需要先getConfigRepository初始化配置实例，只不过Lion的静态方法内部自己初始化了

代码块

Java

```
// 初始化项目配置appkey (META-INF/app.properties配置的app.name值)、默认分组的配置实例
```

```
1 ConfigRepository config = Lion.getConfigRepository();
2 // 初始化指定appkey、默认分组的配置实例
3
4 ConfigRepository config = Lion.getConfigRepository("${appkey}");
5 // 初始化指定appkey、指定分组的配置实例
6 ConfigRepository config = Lion.getConfigRepository("${appkey}", "${group}");
```

1.1、ConfigRepository

代码块

```
1 // 程序中“lion-demo”为appkey, “test.group”为key
2 ConfigRepository config = Lion.getConfigRepository("lion-demo");
3 String value = config.get("test.group");
4 //-----
5 // 获取项目: lion-demo、分组: 默认下所有配置
6 ConfigRepository repository = Lion.getConfigRepository("lion-demo");
7 Map<String, String> configs = repository.getConfigs(); // 获取所有配置 name -> value
8 Set<String> names = repository.getConfigNames(); // 获取所有配置 name
```

1.2、Lion静态方法

代码块

```
1 public class Lion {
2     /** 获取配置API */
3     /**
4      * 方法参数定义:
5      * @param appkey 项目
6      * @param name 配置名
7      * @param defaultValue 配置默认值
8      */
9     // 获取配置值, 如果配置不存在返回null
10    public static String getString(String appkey, String name);
```

```
1 // 获取配置值, 如果配置不存在返回defaultValue
2
3 public static String getString(String appkey, String name, String defaultValue);
4
5 public static Byte getByte(String appkey, String name);
6
7 public static Byte getByte(String appkey, String name, Byte defaultValue);
8
9 public static Short getShort(String appkey, String name);
10
11 public static Short getShort(String appkey, String name, Short defaultValue);
12
13 public static Integer getInt(String appkey, String name);
14
15 public static Integer getInt(String appkey, String name, Integer defaultValue);
16
17 public static Long getLong(String appkey, String name);
18
19 public static Long getLong(String appkey, String name, Long defaultValue);
20
21 public static Float getFloat(String appkey, String name);
22
23 public static Float getFloat(String appkey, String name, Float defaultValue);
24
25 public static Double getDouble(String appkey, String name);
26
27 public static Double getDouble(String appkey, String name, Double defaultValue);
28
29 public static Boolean getBoolean(String appkey, String name);
30
31 public static Boolean getBoolean(String appkey, String name, Boolean defaultValue);
32
33 public static List<String> getList(String appkey, String name);
```

```
45 public static <T> List<T> getList(String appkey, String name, Class<T> clazz);
46
47 public static <T> List<T> getList(String appkey, String name, Class<T> clazz, List<T> defaultValue);
48
49 public static Map<String, String> getMap(String appkey, String name);
50
51 public static <T> Map<String, T> getMap(String appkey, String name, Class<T> clazz);
52
53 public static <T> Map<String, T> getMap(String appkey, String name, Class<T> clazz, Map<String, T>
defaultValue);
54
55 public static <T> T getBean(String appkey, String name, Class<T> clazz);
56
57 public static <T> T getBean(String appkey, String name, Class<T> clazz, T defaultValue);
58
59 /**监听配置变更API**/
60 /**
61  * 方法参数定义:
62  * @param appkey 项目
63  * @param group 分组, 无分组则填写"default"
64  * @param name 配置名
65  * @param configListener 配置变更listener
66  * @param executor 自定义线程池
67 */
68 public static void addConfigListener(String appkey, String group, ConfigListener configListener);
69
70 public static void addConfigListener(String appkey, String group, ConfigListener configListener, Executors
executor);
71
72 public static void removeConfigListener(String appkey, String group, ConfigListener configListener);
73
74 public static void addConfigListener(String appkey, String group, String name, ConfigListener configListen
```

```

76 public static void addConfigListener(String appkey, String group, String name, ConfigListener configListen
ExecutorService executor);
77
78 public static void removeConfigListener(String appkey, String group, String name, ConfigListener configLis
79 }

```

监听配置变更

现在一般使用如上的方法

2.1 默认线程池

```

代码块
1 //监听指定appkey、group下的配置变更
2 ConfigRepository config = Lion.getConfigRepository(${appkey}, ${group});
3 config.addConfigListener(new ConfigListener() {
4     @Override
5     public void configChanged(ConfigEvent configEvent) {
6         //handle event
7     }
8 });
9 //appkey group key listener
10 Lion.addConfigListener("technician-mobile-web","default","technician-mobile-web.tech.resume.share.tips", new
ConfigListener() {
11     @Override
12     public void configChanged(ConfigEvent configEvent) {
13         tipsList = Lion.getList("technician-mobile-web.tech.resume.share.tips", String.class, Lists.newArrayList
美","宜梳妆","宜打扮",
14         "宜变身","宜变换造型","宜出门","宜偶遇","宜突发奇想","宜拥抱变化","宜约会","宜换妆","宜靓颜","宜告白","宜逛街"));
15     }

```

2.2 自定义线程池

代码块

Java

```
1 //监听指定appkey、group下的配置: lion-demo.test.group的变更
2 ConfigRepository config = Lion.getConfigRepository(${appkey}, ${group});
3 //自定义线程池
4 ExecutorService executor = Executors.newFixedThreadPool(2);
5 config.addConfigListener("lion-demo.test.group", new ConfigListener() {
6     @Override
7     public void configChanged(ConfigEvent configEvent) {
8         //handle event
9     }
10 }, executor);
11
12 ExecutorService executor = Executors.newFixedThreadPool(10);
13 Lion.addConfigListener("technician-mobile-web", "default", "technician-mobile-web.tech.resume.techIdList", new
14     ConfigListener() {
15         @Override
16         public void configChanged(ConfigEvent configEvent) {
17             techIdList = Lion.getList("technician-mobile-web", "technician-mobile-web.tech.resume.techIdLi
18 Integer.class);
19         }
20 }, executor);
```

3.2、spring集成

spring集成包含xml方式和注解方式

现有一个这个类

代码块

Java

```
1 public class XmlBean {
2
3     private String placeholder;
4
5     public String getPlaceholder() {
6         return placeholder;
7     }
8
9     public void setPlaceholder(String placeholder) {
10         this.placeholder = placeholder;
11     }
12 }
```

xml方式

代码块

XML

```
1 <beans xmlns="http://www.springframework.org/schema/beans"
2     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3     xmlns:lion="http://code.dianping.com/schema/lion"
4     xsi:schemaLocation="http://www.springframework.org/schema/beans
5         http://www.springframework.org/schema/beans/spring-beans.xsd
6         http://code.dianping.com/schema/lion http://code.dianping.com/schema/lion/lion-1.0.xsd">
7     <!-- 注入默认appkey (META-INF/app.properties配置的app.name值)、默认分组的配置实例 -->
8     <lion:config/>
9     <!-- 类"XmlBean"、属性"placeholder"、配置key:"test.placeholder"均为特定例子 -->
10    <bean id="xmlBean" class="com.dianping.lion.client.spring.XmlBean">
11        <property name="placeholder" value="${test.placeholder}"/>
12    </bean>
```

</beans>

<lion:config /> 可以指定属性:

1. propertiesPath 用于指定自定义的配置文件的位置, 默认值是config/applicationContext.properties。applicationContext.properties 有两个功能:
 - a. 配置覆盖: 配置覆盖只在dev和alpha环境有效, 其它环境自动失效, 除非在 Lion 初始化时指定includeLocalProps=true 比如在配置文件中指定:lion-test.local=local 可以用上面的 key-value 覆盖dev环境开发时 Lion 系统上的配置, 可以在本地开发调试时自己定义自己的配置信息, 在上其他测试环境和生产环境时自动失效, 避免忘记删除此配置导致其他环境使用自定义配置信息。
 - b. 配置引用: 配置引用在所有环境中有效, 比如在配置文件中指定:lion-test.local=\${lion-test.remote} 这时在 Spring 配置文件中遇到占位符 \${lion-test.local}时, 会自动使用\${lion-test.remote}的配置
2. includeLocalProps 用于指定是否需要在线上环境使用自定义配置文件中的值, 默认值是 false
3. order 用于指定 Lion 配置的加载顺序, 可用来确定spring其他配置加载器的覆盖顺序
4. ignoreUnresolvablePlaceholders 用于指定当lion加载不到配置时, 是否使用spring中定义的其他配置解析器继续加载配置
5. appkey group

注解方式

- @LionConfigRepositoryListener 注册配置变更监听
- @LionConfig 获取配置

代码块

Java



```
1 //监听appkey、group分组下任意配置的变更
2 @LionConfigRepositoryListener(appkey = "appkey", group = "group")
3 public class ConfigRepositoryListener implements ConfigListener {
4     @Override
5     public void configChanged(ConfigEvent configEvent) {
6         if(configEvent.getKey()=="xxxxx")xxxxxx
7         System.err.println("config event====" + configEvent);
8     }
}
```

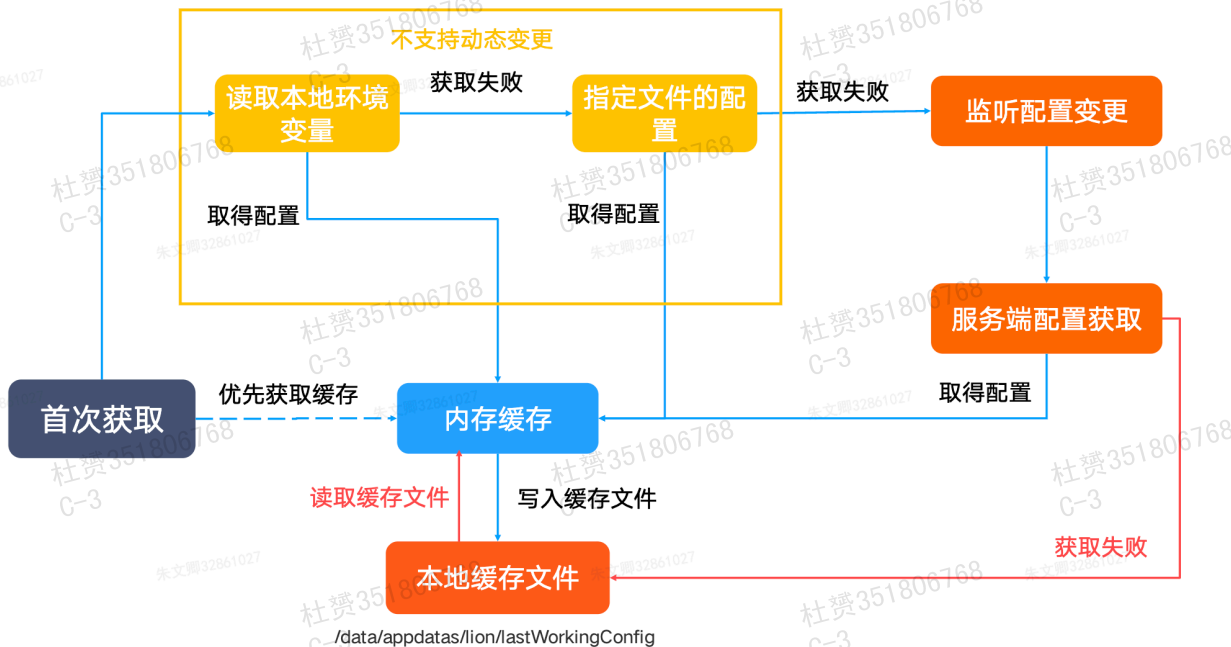
```
10 }
11 //监听appkey、group分组下某个配置的变更
12 @LionConfigRepositoryListener(appkey = "appkey", group = "group", name = "name")
13 public class ConfigRepositoryListener implements ConfigListener {
14     @Override
15     public void configChanged(ConfigEvent configEvent) {
16         System.err.println("config event====" + configEvent);
17     }
18 }
19 //不要在@Configuration使用@LionConfig注解, 否则可能会导致注解不生效
20 @Component
21 public class ConfigHolder {
22     // 省略了get/set方法, 业务接入时自行补充
23     //从Lion获取配置, 同时监听配置变更, 未指定appkey的key需要在创建时加appkey前缀
24     @LionConfig("lion-test.remote")
25     private String remote;
26     //只支持从Lion获取配置, 但不监听, 不会更新配置, 未指定appkey的key需要在创建时加appkey前缀
27     @Value("lion-test.suzhou")
28     private String multi;
29     //lion-client 0.10.9.2+支持主动设置appkey&group; 0.11.0+ 版本支持converter可以自定义, revertOriginal=true表示
30     //删除后使用默认值
31     @LionConfig(appkey = "lion-demo", key = "remoteMap", converter = DefaultJsonConfigConvert.class, revertOri
32     true)
33     private Map<String, Object> remoteMap;
34     //自定义converter, 处理复杂类型
35     @LionConfig(appkey = "lion-demo", key = "lion-demo.wjh-test", converter = UserCustomConvert.class)
36     private Map<String, List<Person>> complicateMap;
```

```
40 //自定义Convert
41
42 public class UserCustomConvert extends AbstractConfigConvert {
43     @Override
44     public Object convertStringToObject(String json, Class<?> type) {
45         try {
46             return JsonUtils.fromJson(json, new TypeReference<Map<String, List<Person>>>(){});
47         } catch (IOException e) {
48             Cat.logError("Lion.convert.failed", e);
49         }
50         return json;
51     }
52 }
53 }
```

原理

Lion 设计与实现

配置优先级（get源码分析）



1. **系统变量(环境变量)获取**, 首先读取系统变量(环境变量)的值, 如果获取到值则缓存到内存并返回, 值保持不变。系统变量(环境变量)中的Key, 不从Lion服务端获取配置, 同时不支持监听配置能力。
2. **Lion服务端配置获取**, 系统变量(环境变量)未能获取配置, 则从Lion服务端获取配置, 并保持配置监听能力。
3. **本地缓存文件获取**, Lion服务端获取失败, 则从本地缓存文件获取配置, 并保持配置监听能力。在监听到服务端到配置变更后, 客户端同步更新内存缓存和本地文件缓存。本地缓存文件地址: /data/appdatas/lion/lastWorkingConfig/\${appkey}/\${appkey}+\${group}+\${key}。
4. 通过下面的源码分析可以看出来Lion配置加载的顺序: **加载外部文件->内存缓存->环境变量->系统变量->Lion服务端配置->本地文件缓存**

测试配置加载顺序

Java

```
1 package com.yb0os1.lion;
2
3 import com.dianping.lion.client.Lion;
4
5 /**
6  * Lion 配置获取优先级分析
7  */
```

```

8 public class LionPriorityAnalysis {
9     public static void main(String[] args) {
10         // 示例: 如何验证 Lion 配置获取优先级 前提: lion服务端设置了11111
11
12         // 1. 设置系统变量, 验证系统变量优先级高于 Lion 服务端
13         //System.setProperty("test", "100");
14         // 此时 Lion.getInt("carnation-campaign-web", "test") 应该返回 100
15         //System.out.println(Lion.getInt("carnation-campaign-web", "test")); //100
16         // 2. 设置环境变量 (需要在启动前设置)
17         //export test=200
18         // 此时如果同时存在系统变量和环境变量, 环境变量优先级更高
19         // System.out.println(Lion.getInt("carnation-campaign-web", "test")); //200
20         // 3. 没有设置系统变量和环境变量
21         // 此时 Lion.getInt("carnation-campaign-web", "test") 将直接从 Lion 服务端获取
22         System.out.println(Lion.getInt("carnation-campaign-web", "test"));
23     }
24 }

```

^ get时候的源码分析

Java

/ 每一个get都涉及到doGet 也就是最底层的调用

```

protected String doGet(String appkey, String group, String name, ConfigContext context) {
    name = KeyUtils.fixKey(name);

    //1、支持从外部文件获取配置, 通过ConfigCache#loadProperties 加载外部配置文件 现在已经废弃
    //。配置想实现动态变更, 只要修改远程文件, 分批重启机器或定时重新 load 文件
    if (this.userProvidedConfig != null) {
        String value = (String) this.userProvidedConfig.get(name);
        if (value != null) {
            return value;
        }
    }
}

```

ConfigValue configValue = null;

context = context == null ? this.defaultContext : context;

//2、如果启用了缓存, 从缓存中获取配置 默认启用缓存

```
if (context.isUseCache()) {  
    configValue = this.readCache(appkey, group, name);  
}
```

//3、没有启用缓存, 从配置源查找

```
if (configValue == null) {  
    //Lion 通过 ConcurrentMap<String, ConfigFuture> configFutures 给并发请求分配配置加载任务,  
    // 使得仅有一个线程在执行配置加载的任务, 其他线程等待该线程执行并返回执行结果  
    ConfigFuture<ConfigValue> existingFuture = null;  
    String uniqueKey = StringUtils.combine(new Object[]{appkey, group, name});  
    // 新建一个配置获取任务  
    ConfigFuture future = new ConfigFuture(uniqueKey);  
    // putIfAbsent 的效果: !map.containsKey(key) : map.put(key, value) ? map.get(key);  
    // 即: 在【任务表】中, 若没有任务, 则填充任务; 若已有任务, 那么获取该任务  
    existingFuture = (ConfigFuture)this.configFutures.putIfAbsent(uniqueKey, future);  
    if (existingFuture != null) {  
        // 如果之前有任务在执行了, 那么其他线程阻塞等待获取结果  
        try {  
            //// 阻塞等待获取结果  
            configValue = (ConfigValue)existingFuture.get((long)metaConfigManager.getPollTimeout(),  
imeUnit.MILLISECONDS);  
        } catch (ExecutionException e) {  
            throw new LionException(e.getCause());  
        } catch (Exception e) {  
            throw new LionException(e);  
        }  
    } else {  
        try {//// 执行任务, 执行成功后通知其他线程获取配置  
            // 调用到这里如下图 依次遍历查找 执行任务: 加载本地/ 远程的配置  
            // 这一行是重点  
            configValue = this.configLoaderManager.get(appkey, group, name, context);  
        }  
    }  
}
```

// 塞值 并 通知其他线程

```
future.setResult(configValue);  
if (configValue != null && context.isUseCache()) {  
    // 更新内存缓存 (loadedConfig), (可选: 持久化到本地)  
    this.writeCache(appkey, group, name, configValue, true);  
}  
} catch (Throwable e) {  
    future.setResult(e);  
    throw new LionException(e);  
} finally {  
    this.configFutures.remove(uniqueKey);  
}  
}  
}
```

```
Governor.usedGovern(appkey, group, name, configValue);  
return configValue.getValue();  
}
```

/ConfigFuture

```
public class ConfigFuture<T> implements Future {  
    private String key;  
    private CountDownLatch latch;  
    private Object result;  
    // ...  
    @Override  
    public T get() throws InterruptedException, ExecutionException {  
        // 通过 countdownlatch 来阻塞等待  
        latch.await();  
        if (result instanceof Throwable) {  
            throw new ExecutionException((Throwable) result);  
        }  
        return (T) result;  
    }  
}
```

```
public void setResult(Object result) {
```

```
    this.result = result;
```

```
    // 注意: 这里有个 countDown, 意味着 setResult 之后, get() 方法就不会再阻塞
```

```
    latch.countDown();
```

```
// 写缓存的函数
```

```
private void writeCache(String appkey, String group, String name, ConfigValue newConfig, boolean isForce) {
```

```
    if (!isForce && ConfigUtils.isNullConfig(newConfig)) {
```

```
        String namespace = StringUtils.combine(new Object[]{appkey, group});
```

```
        Map<String, ConfigValue> loadedConfig = (Map)this.cacheConfig.get(namespace);
```

```
        if (CollectionUtils.isEmpty(loadedConfig)) {
```

```
            loadedConfig.remove(name);
```

```
        }
```

```
        if (this.enablePersist) {
```

```
            TaskManager.submit(new PersistTask(appkey, group, name, (String)null));
```

```
        }
```

```
    } else { // 上面的代码一直走else
```

```
        // 逻辑就是先获取内存缓存, 不存在直接写入内存; 存在的话看看是否一致 一致不变 不一致修改为新值
```

```
        // 持久话进行判断 一般都是进行了持久话
```

```
        String namespace = StringUtils.combine(new Object[]{appkey, group});
```

```
        ConcurrentMap<String, ConfigValue> loadedConfig = (ConcurrentMap)this.cacheConfig.get(namespace);
```

```
        if (loadedConfig == null) {
```

```
            loadedConfig = new ConcurrentHashMap();
```

```
            this.cacheConfig.put(namespace, loadedConfig);
```

```
        }
```

```
        ConfigValue oldConfig = (ConfigValue)loadedConfig.get(name);
```

杜赞351806768 C-3

```
        if (this.needWriteCache(newConfig, oldConfig, true)) {
            loadedConfig.put(name, newConfig);
        }

        if (this.enablePersist && !ConfigUtils.isNotChanged(newConfig, oldConfig) &&
newConfig.isLastWorkingConfig()) {
            TaskManager.submit(new PersistTask(appkey, group, name, newConfig.getValue()));
        }
    }
}
```

杜赞351806768 C-3

//configLoaderManager.get核心代码
//this.configLoaders有三个元素，并且位置固定（通过Order注解控制） 分别是针对环境变量/ 系统变量、Lion远程、本地缓存
// Collections.sort(configLoaders, OrderComparator.INSTANCE);

```
for(ConfigLoader configLoader : this.configLoaders) {
    try {
        if (configLoader.isEnabled() && context.isValidSource(configLoader.getName())) {
            //configLoader有三种实现，分别是针对环境变量/ 系统变量、Lion远程、本地缓存
            value = configLoader.get(appkey, group, name);
            if (value != null) {
                value.setSource(configLoader.getName());
                break;
            }
        }
    }
}
```

杜赞351806768 C-3

//先看针对环境变量/系统变量的实现PropertiesFileConfigLoader 列表第0位

```
public ConfigValue get(String appkey, String group, String name) throws Exception {
    String value = System.getenv(key);//先获取环境变量
    if (value != null) {
        return new ConfigValue(key, value, this.getName());
    } else {
```

```
value = System.getProperty(key); // 再获取系统变量
if (value != null) {
    return new ConfigValue(key, value, this.getName());
} else { // 从内存配置文件读取
    return this.properties != null && this.properties.containsKey(key) ? new ConfigValue(key,
his.properties.getProperty(key), this.getName()) : null;
}
}
```

/再看针对Lion服务器配置的HttpConfigLoader 列表第1位

```
public ConfigValue get(String appkey, String group, String name) throws Exception {
    ConfigValue result = null;
    // this.appName不为空且配置名称是组件键
    if (this.appName != null && KeyUtils.isComponentKey(name)) {
        // 获取应用级配置 针对特定应用的配置
        ConfigValue acv = this.doGet(appkey, group, name);
        // 获取平台级配置 可能是全局共享的配置, 通过应用名前缀区分
        ConfigValue pcv = this.doGet(this.appName, group, this.appName + "." + name);
        // 优先使用平台级配置, 如果平台级配置为空则使用应用级配置
        result = ConfigUtils.isNullConfig(pcv) ? acv : pcv;
    } else {
        result = this.doGet(appkey, group, name);
    }
    return result;
}
```

/*上面的doGet做的事情有:

1通过repositoryManager获取指定应用(appkey)的项目仓库(ProjectRepository)

2从项目仓库中获取指定分组(group)和名称(name)的配置值

3返回获取到的配置值

```
private ConfigValue doGet(String appkey, String group, String name) throws Exception {
    ProjectRepository repository = this.repositoryManager.getProjectRepository(appkey);
    ConfigValue cv = repository.get(group, name);
}
```

杜赞351806768 C-3

```
    }  
    return cv;  
}  
  
/ 最后看一下针对本地文件缓存的LastWorkingConfigLoader 列表第2位  
public ConfigValue get(String appkey, String group, String name) throws Exception {  
    if (this.enableLastWorking) {  
        try {  
            //APPKEY_GROUP_SEPARATOR 是+号 也就是拼接 appkey+group+name  
            String fileName = StringUtils.join(ConfigConstants.APPKEY_GROUP_SEPARATOR, new Object[]{appkey,  
group, name});  
            //persistDir就是/data/appdatas/lion/lastWorkingConfig/appkey/filename  
            String content = FileUtils.read(this.persistDir + "/" + FileUtils.checkFileName(fileName));  
            //如果name是以password结尾尝试加密  
            if (KeyUtils.isSecretKey(name)) {  
                content = CodecUtils.tryDecode(content);  
            }  
            //读取到就返回 否则捕获异常返回null  
            return new ConfigValue(name, content, this.getName());  
        } catch (FileNotFoundException var6) {  
            return null;  
        }  
    } else {  
        return null;  
    }  
}
```

杜赞351806768 C-3

0 个用法

```
public ConfigValue get(String appkey, String group, String name, ConfigContext context) throws Exception {
    ConfigValue value = null;
    for (ConfigLoader configLoader : this.configLoaders) {
        try {
            if (configLoader.isEnabled() && context != null) {
                value = configLoader.get(appkey, group, name, context);
                if (value != null) {
                    value.setSource(configLoader);
                    break;
                }
            }
        } catch (Exception e) {
            if (configLoader == this.lastConfigLoader) {
                throw e;
            }
        }
    }
}
```

控制台

对表达式求值(🔗)或添加监视(🔗)

> this = {ConfigLoaderManager@3652}

> appkey = "carnation-campaign-web"

> group = "default"

> name = "test"

> context = {ConfigContext@3650}

> value = {ConfigValue@4015} "ConfigResult[key=carnation-campaign-web.test,value=11111,tag=,source=null,modifyTime=1754980838831,releaseTime=1754980838831]"

> configLoader = {HttpConfigLoader@3723} "HTTP"

> this.lastConfigLoader = {LastWorkingConfigLoader@3699} "LAST_WORKING"

设置值 F2 创建呈现器

configLoader: "HTTP" configLoaders: size = 3

this.configLoaders = {ArrayList@3698} size = 3

- 0 = {PropertiesFileConfigLoader@3702} "LOCAL" 系统变量/环境变量
 - environment = "test"
 - propertiesPath = null
 - includeLocalProps = null
 - properties = {Properties@3708} size = 0
 - logger = {Log4jLogger@3709}
 - configLoaderListeners = {ConcurrentHashMap@3710} size = 1
 - enabled = true
- 1 = {HttpConfigLoader@3723} "HTTP" Lion服务端配置
 - metaConfig = {MetaConfigManager@3662}
 - repositoryManager = {ConfigRepositoryManager@4017}
 - appName = "carnation-campaign-web"
 - projectManagers = {ConcurrentHashMap@4019} size = 0
 - usedConfigs = {Collections\$SetFromMap@4020} size = 0
 - syncExecutor = {ThreadPoolExecutor@4021} "java.util.concurrent.ThreadPoolExecutor@6d24ffa1[Running, pool size = 1, active threads = 0]"
 - monitor = {CatMonitor@4022}
 - logger = {Log4jLogger@4023}
 - configLoaderListeners = {ConcurrentHashMap@4024} size = 1
 - enabled = true
- 2 = {LastWorkingConfigLoader@3699} "LAST_WORKING" 本地文件缓存
 - persistDir = "/data/appdatas/lion/lastWorkingConfig/carnation-campaign-web"
 - enableLastWorking = false
 - logger = {Log4jLogger@4029}
 - configLoaderListeners = {ConcurrentHashMap@4030} size = 1
 - enabled = true

针对上面源码的问题或思考:

1. 涉及到了内存缓存, 那么缓存的写入、更新、过期是在哪些场景下触发, 以及是如何执行的呢?

代码块

```
1 package com.yb0os1.lion;
2
3 import com.dianping.lion.client.ConfigEvent;
4 import com.dianping.lion.client.ConfigListener;
```

```

5 import com.dianping.lion.client.Lion;
6 import org.springframework.stereotype.Component;
7
8 @Component
9 public class ConfigHodler {
10     private static int test;
11
12     static {
13         Lion.addConfigListener("carnation-campaign-web", "default", "test", new ConfigListener() {
14             @Override
15             public void configChanged(ConfigEvent configEvent) {
16                 test = Lion.getInt("carnation-campaign-web", "test");
17             }
18         });
19     }
20     public static void main(String[] args) throws InterruptedException {
21         // 环境变量设置了为666
22         test = Lion.getInt("carnation-campaign-web", "test");
23         while (true) {
24             System.out.println(test);
25             Thread.sleep(1000);
26         }
27     }
28 }
29 // 一开始输出666。当Lion服务端配置改变为519 就输出为519。推测当监听器监听到Lion服务端的变化时 会删除内存缓存 并且将新值加入缓存

```

2.缓存的大小是否需要进行控制？

3.当缓存中无配置时，需从本地或远程去实时加载配置，那么有如下几个问题需要解决。

1、同时有大量获取同一配置的请求来临时如何处理配置的实时加载？

2、有哪些加载配置的形式？



ConfigFuture 利用 [CountDownLatch](#) 对多线程间 wait/notify 操作的良好封装，实现同一配置值被多线程阻塞等待获取。详细可以看代码

ConcurrentMap 则保证了并发情况下任务的唯一性，可以说是维护了一个唯一任务表。这一个设计很巧妙

Lion 在整个请求过程中做了较多性能和并发上的优化，使用内存 Cache 和 本地文件保障了读取时的性能和稳定性，CountDownLatch、阻塞队列、原子类、Sync 等并发组件保证并发时的数据读写安全。

内部大佬源码分析

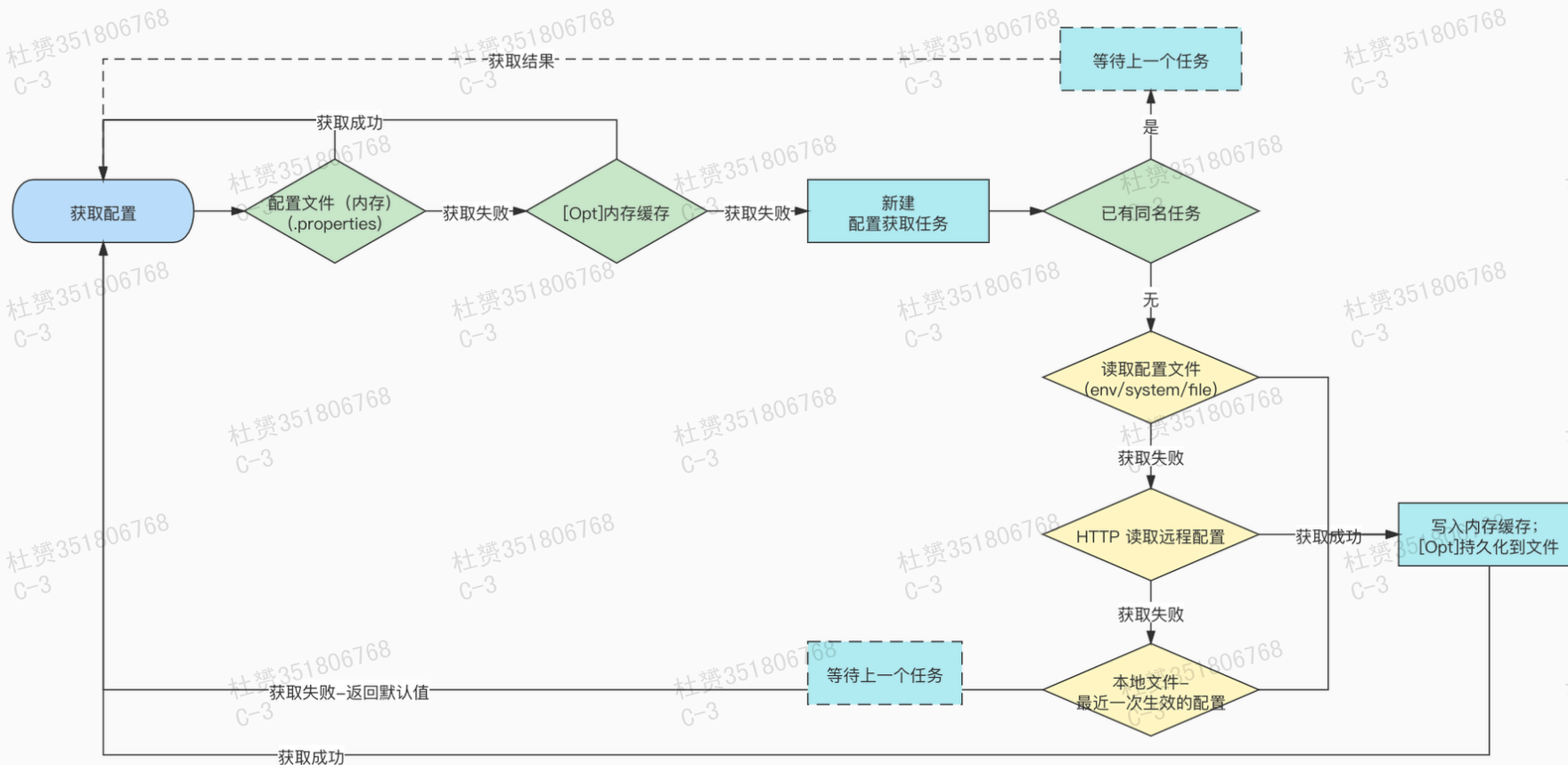
前言

本文基于 lion-client 0.8.20 Java 源码，主要讲述 Lion 客户端获取配置的过程及其中一些优秀设计。

核心围绕 Lion#get 这一方法入口，代码中涉及配置Project、Group 逻辑不做额外描述会进行省略或精简，Lion 背景、使用、架构相关可参考附录一。

一、Lion Client 配置读取流程图

Lion · 客户端配置获取流程图（简化）



二、流程详解

1、从配置文件中获取配置

- 需主动提前通过 ConfigCache#loadProperties 加载外部配置文件到内存中，后续优先从配置文件从获取配置。

ConfigCache#loadProperties 加载外部配置文件：

代码块

Java

```
1 public class ConfigCache {
```

```

1 // 需手动执行从外部加载
2
3 public void loadProperties(String propertiesFile) throws IOException {
4     // 支持 classpath 和 FileSystem 两种 load 形式
5     Properties props = PropertiesUtils.load(propertiesFile);
6     if (props != null) {
7         setPts(props);
8     }
9 }
10
11 public void setPts(Properties properties) {
12     if (properties != null) {
13         Map<String, String> map = new HashMap<String, String>();
14         for (String key : properties.stringPropertyNames()) {
15             map.put(key, properties.getProperty(key));
16         }
17         configManager.setProperties(map);
18     }
19 }
20 }

```

ConfigManager#doGet 获取配置 - 从配置文件中获取:

代码块

Java

```

1 public class ConfigManagerImpl {
2
3     protected String doGet(String key, String defaultValue, ConfigContext context) {
4         key = KeyUtils.fixKey(key);
5         // 1、扩展点，支持从外部文件获取配置
6         if (userProvidedConfig != null) {
7             String value = userProvidedConfig.get(key);
8             if (value != null) {
9                 return value;

```

```
10 }
11 }
12 //...
13 }
14 }
```

思考:

从简单意义上来说, ConfigCache#loadProperties 加载远程文件, 是否就已经实现了一个 MVP 的 Lion 配置?

配置想实现动态变更, 只要修改远程文件, 分批重启机器或定时重新 load 文件。

2、从内存缓存中获取配置

Lion 默认开启内存缓存, 开启后优先从 loadedConfig 内存缓存中获取配置。缓存的读取过程如下代码所示。

ConfigManager#doGet 获取配置 - 从缓存中获取:

代码块

```
1 public class ConfigManagerImpl {
2
3     // 配置缓存
4     private Map<String, ConfigValue> loadedConfig = new ConcurrentHashMap<String, ConfigValue>();
5
6     protected String doGet(String key, String defaultValue, ConfigContext context) {
7         // ...
8         // 2、从内存缓存中获取配置
9         ConfigValue configValue = null;
10        context = (context == null ? defaultContext : context);
11        if (context.isUseCache()) {
12            // 默认都是允许从缓存中读取配置的
13            configValue = readCache(key);
14        }
15        // ...
16    }
17 }
```

```
16         return configValue.getValue() == null ? defaultValue : configValue.getValue();
17     }
18
19     private ConfigValue readCache(String key) {
20         return loadedConfig.get(key);
21     }
22 }
```

思考：

- 既然涉及到缓存，那么缓存的写入、更新、过期是在哪些场景下触发，以及是如何执行的呢？
- 缓存的大小是否需要控制？

3、从本地/网络读取配置

当缓存中无配置时，需从本地或远程去实时加载配置，那么有如下几个问题需要解决。

- 1、同时有大量获取同一配置的请求来临时如何处理配置的实时加载？
- 2、有哪些加载配置的形式？

3.1 新建配置加载任务

简述：通过 `ConcurrentMap<String, ConfigFuture> configFutures` 给并发请求分配配置加载任务，使得仅有一个线程在执行配置加载的任务，其他线程等待该线程执行并返回执行结果。

代码解析更容易理解此过程。

代码解析更容易理解此过程。

ConfigManager#doGet 获取配置 - 新建任务加载配置：



```
1 public class ConfigManagerImpl extends AbstractConfigManager implements ConfigLoaderListener {
2     // 配置获取任务表
3     private ConcurrentMap<String, ConfigFuture> configFutures = new ConcurrentHashMap<String, ConfigFuture>(
4
5     protected String doGet(String key, String defaultValue, ConfigContext context) {
6         // ...
7         ConfigFuture<ConfigValue> existingFuture = null;
8         // 新建一个配置获取任务
9         ConfigFuture future = new ConfigFuture(key);
10        // putIfAbsent 的效果: !map.containsKey(key) : map.put(key, value) ? map.get(key);
11        // 即: 在【任务表】中, 若没有任务, 则填充任务; 若已有任务, 那么获取该任务
12        existingFuture = configFutures.putIfAbsent(key, future);
13        if (existingFuture != null) {
14            // 如果之前有任务在执行了, 那么其他线程阻塞等待获取结果
15            try {
16                // 阻塞等待获取结果
17                configValue = existingFuture.get(metaConfigManager.getPollTimeout(), TimeUnit.MILLISECONDS);
18            } catch (Exception e) {
19                // ... 异常处理
20            }
21        } else {
22            // 执行任务, 执行成功后通知其他线程获取配置
23            try {
24                // 执行任务: 加载本地/ 远程的配置
25                configValue = configLoaderManager.get(key, context);
26                // 塞值 并 通知其他线程
27                future.setResult(configValue);
28                if (configValue != null && context.isUseCache()) {
29                    // 更新内存缓存 (loadedConfig), (可选: 持久化到本地)
30                    writeCache(key, configValue, true);
31                }
32            }
```

```
32     } catch (Throwable e) {
33         future.setResult(e);
34         throw new LionException(e);
35     } finally {
36         // 删除任务
37         configFutures.remove(key);
38     }
39 }
40 // ...
41 return configValue.getValue() == null ? defaultValue : configValue.getValue();
42 }
43 }
```

```
44
45 public class ConfigFuture<T> implements Future {
46     private String key;
47     private CountDownLatch latch;
48     private Object result;
49     // ...
50     @Override
51     public T get() throws InterruptedException, ExecutionException {
52         // 通过 countdownlatch 来阻塞等待
53         latch.await();
54         if (result instanceof Throwable) {
55             throw new ExecutionException((Throwable) result);
56         }
57         return (T) result;
58     }
59
60     public void setResult(Object result) {
61         this.result = result;
62         // 注意: 这里有个 countdown, 意味着 setResult 之后, get() 方法就不会再阻塞
63         latch.countDown();
64     }
```

思考：

为什么要用 `putIfAbsent`，并且每次都要 `new ConfigFuture`？写成简单的 `[if contains ? do get : new and put ;]` 不是更好吗？

3.2 配置加载管理类

引入 `ConfigLoaderManager` 配置加载器管理类，维护多种加载器的初始化，封装通过多种加载器加载配置的操作。

`ConfigLoaderManager#get`：多种加载器加载配置操作的封装

代码块

```
1 public class ConfigLoaderManager {
2     /*
3     * 配置加载器：默认实现会有按序如下三种：
4     * PropertiesFileConfigLoader
5     * HttpConfigLoader
6     * LastWorkingConfigLoader
7     */
8     private List<ConfigLoader> configLoaders;
9
10    public ConfigValue get(String appkey, String group, String name, ConfigContext context) throws Exception
11        ConfigValue value = null;
12    for (ConfigLoader configLoader : configLoaders) {
13        try {
14            // ... configLoader 有效性校验
15            // 从配置加载器中获取配置
16            value = configLoader.get(appkey, group, name);
```

```

17         if (value == null) {
18             // 没有就下一个
19             continue;
20         }
21         // 标记当前配置的数据源是哪个配置加载器
22         value.setSource(configLoader.getName());
23         break;
24     } catch (Exception e) {
25         // ... 异常处理
26     }
27 }
28 // ... 空值处理、日志等非核心操作
29 return value;
30 }
31 }

```

3.3 PropertiesFileConfigLoader：从本地配置文件中加载配置

属性文件配置加载器，允许从环境变量、系统配置、本地文件加载配置。

每个 ConfigLoader 都有 `@Order` 注解，用于声明其执行顺序。

PropertiesFileConfigLoader#get:

代码块

Java

```

1  @Order(0)
2  public class PropertiesFileConfigLoader extends AbstractConfigLoader {
3      // 从其工厂类中获取 PropertiesFileConfigLoader 实例时，会去做配置的初始化。
4      // 详见 PropertiesFileConfigLoaderFactory#getConfigLoader
5      private Properties properties = new Properties();

```

```

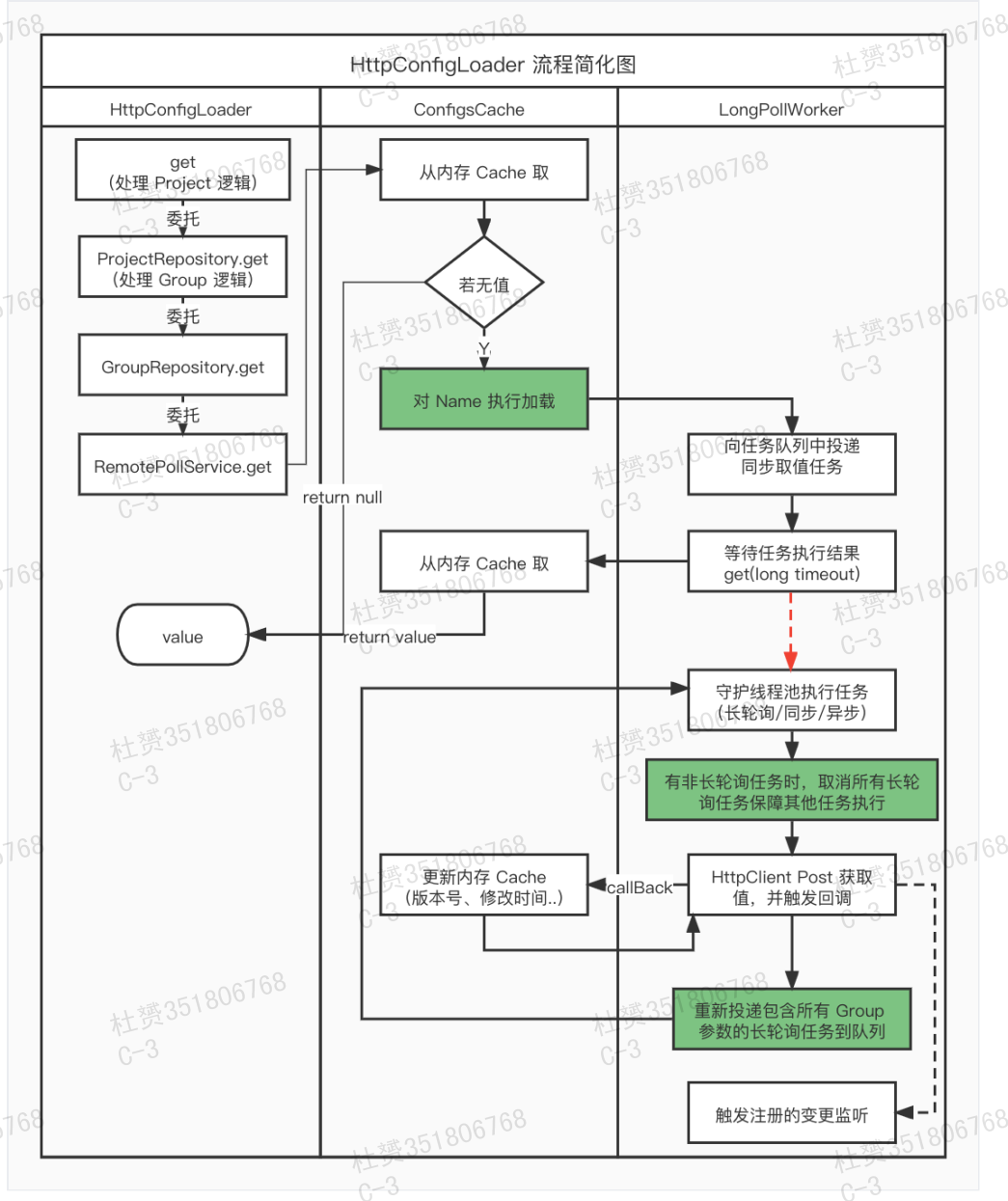
6  //...
7  @Override
8  public ConfigValue get(String appkey, String group, String name) throws Exception { //FIXME 分组配置
9      // 0、从环境变量读取
10     String value = System.getenv(key);
11     if (value != null) {
12         return new ConfigValue(key, value, getName());
13     }
14     // 1、从系统配置读取
15     value = System.getProperty(key);
16     if (value != null) {
17         return new ConfigValue(key, value, getName());
18     }
19     // 2、从内存配置文件读取
20     if (properties != null && properties.containsKey(key)) {
21         return new ConfigValue(key, properties.getProperty(key), getName());
22     }
23     return null;
24 }
25 //...
26 }

```

3.4 HttpConfigLoader: 从远程加载配置

HttpConfigLoader 的实现较为复杂, 复杂性主要围绕着 Project/Group 取数兼容 (冗长的委托取值)、长轮询的 Http 请求、结合本地缓存并发获取配置。

以下流程简化图描述了 HttpConfigLoader 在执行取值过程中的主要行为:



Point 1、执行流程中核心操作对象是内存 Cache

从上图可见, HttpConfigLoader 的直接数据源是 ConfigsCache 的内存 Cache。当未获取到配置, 也是先加载配置, 再从内存 Cache 中读取。在实际系统运行过程中, 除了一开始的配置初始化会触发配置的加载, 之后**很少**会触发配置的重新加载。即使有配置的变化, Lion 也会通过**长轮询机制主动推送**到客户端并更新至 Cache 中。

在 HttpConfigLoader 主动加载配置的过程中有较多的长轮询请求处理，其主要目的是为了不让长轮询请求阻塞住正常的 get (sync/async) 请求：

代码块

```
1 public synchronized <T> T get(AbstractPollParams params, PollCallback callback) throws Exception {
2     //...
3     PollFuture request = currentRequest;
4     // 若为长轮询请求，则主动取消
5     if (request != null && request.getCallback().getPollType() == PollTypeEnum.LONG) {
6         logger.info("aborting long poll: " + request.getPollParams());
7         request.cancel(true);
8     }
9     try {
10         T result = (T) future.get(metaConfig.getPollTimeout(), TimeUnit.MILLISECONDS);
11         return result;
12     } catch (ExecutionException e) {
13         //... 异常处理
14     }
15 }
16
17 private void doPoll() throws InterruptedException {
18     //... 等待 50 ms, 若50ms内无请求，才执行长轮询请求
19     if (PollTypeEnum.isLongPollType(currentRequest.getCallback())) {
20         TimeUnit.MILLISECONDS.sleep(50);
21         if (pollQueue.size() > 0) {
22             currentRequest = getPollRequest();
23         }
24     }
25     //...
26 }
27
```

```

private PollFuture getPollRequest() throws InterruptedException {
    //... 如果是长轮询请求, 且队列中还有请求, 则丢弃长轮询请求
    while (PollTypeEnum.isLongPollType(pollRequest.getCallback()) && pollQueue.size() > 0) {
        if (logger.isDebugEnabled()) {
            logger.debug("skipped poll request: " + pollRequest.getPollParams());
        }
        // skip long poll request
        pollRequest = pollQueue.poll(1, TimeUnit.SECONDS);
        if (pollRequest == null) {
            return null;
        }
    }
    //...
}

```

Point 3、藏在幕后的长轮询

在所有正常请求处理完成后, 任务处理线程会继续长轮询任务的投递与执行, 以实现长链接获取实时的推送配置变更事件:

代码块

```

1 protected void issueLongPoll() {
2     if (groups.size() > 0) {
3         PollParamsV2 params = new PollParamsV2();
4         for (GroupRepository repository : groups.values()) {
5             // 将所有 Group 请求参数填充到 params
6             repository.assembleLongPollParams(params);
7         }
8         if (!params.isEmpty()) {
9             // 往阻塞队列中投递长轮询请求
10            PollFuture pollRequest = new PollFuture(params, longPollCallback);
11            addPollRequest(pollRequest);
12        }

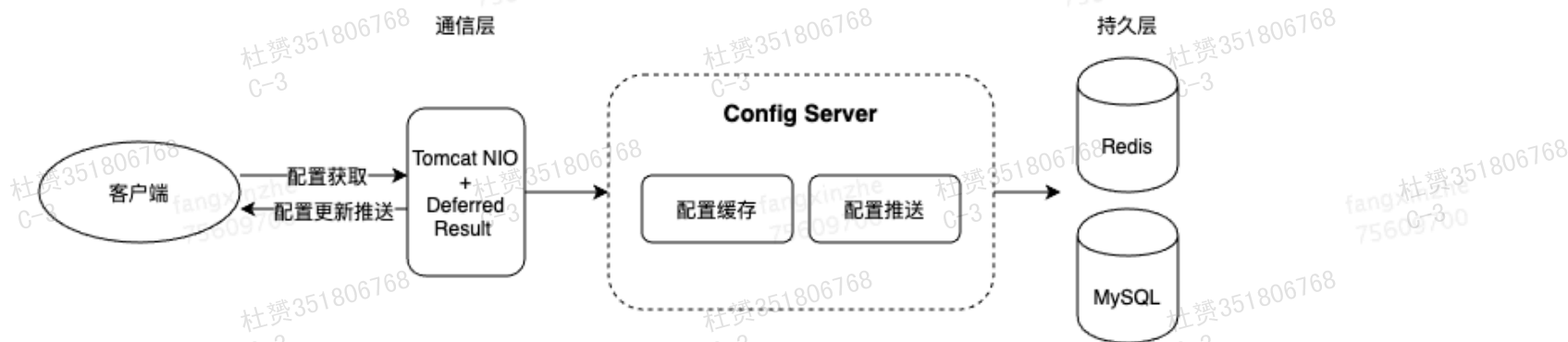
```

长轮询在 Lion 中的使用，不仅是获取配置，而且变相的实现了长连接推送配置变更，可见：[Lion 通信协议方案](#)。不过文中也说了，长轮询这一机制对于不是频繁变更的配置而言是有比较大的服务器资源空耗。

目前lion-client获取配置以及配置变更推送采用HTTP长轮询方式。

- 客户端会发起一个HTTP请求直连Lion Config Server，请求信息包含配置项目、分组或者配置key，以及配置的版本号。
- Config Server监听配置变更，如果超时阈值100s内有客户端关注的配置修改，返回结果给客户端；如果100s内没有配置修改，Config Web会将连接挂起。
- 断开连接后，客户端立即发起下一次长轮询请求。

Config Server当前采取Tomcat NIO、以及Spring DeferredResult实现长轮询，Config Server内存会缓存配置、客户端连接。



3.5 LastWorkingConfigLoader：取最后一次生效的配置

LastWorkingConfigLoader 实现就较为简单暴力，读取对应持久化的磁盘文件，若有就返回

代码块

```
1 public ConfigValue get(String appkey, String group, String name) throws Exception {
2     if(enableLastWorking) {
3         try {
4             String fileName = StringUtils
```

```

5      .join(ConfigConstants.APPKEY_GROUP_SEPARATOR, appkey, group, name);
6      // 读磁盘
7      String content = FileUtils.read(persistDir + "/" + FileUtils.checkFileName(fileName));
8      if (KeyUtils.isSecretKey(name)) {
9          content = CodecUtils.tryDecode(content);
10     }
11     return new ConfigValue(name, content, getName());
12 } catch (FileNotFoundException e) {
13     return null;
14 }
15 } else {
16     return null;
17 }
18 }

```

三、设计思考

1、ConfigFuture 与 ConcurrentMap<String, ConfigFuture> 组成的并发数据获取控制

ConfigFuture 利用 `CountDownLatch` 对多线程间 wait/notify 操作的良好封装，实现同一配置值被多线程阻塞等待获取。

ConcurrentMap 则保证了并发情况下任务的唯一性，可以说是维护了一个唯一任务表。

ConfigFuture 实现：

代码块

Java



```

1 public class ConfigFuture<T> implements Future {
2
3     private String key;

```

```
1 private CountdownLatch latch;  
2 private Object result;  
3  
4  
5  
6  
7 public ConfigFuture(String key) {  
8     this.key = key;  
9     latch = new CountdownLatch(1);  
10 }  
11  
12 @Override  
13 public T get(long arg0, TimeUnit arg1)  
14     throws InterruptedException, ExecutionException, TimeoutException {  
15     latch.await(arg0, arg1);  
16     if(result instanceof Throwable) {  
17         throw new ExecutionException((Throwable)result);  
18     }  
19     return (T) result;  
20 }  
21  
22 public void setResult(Object result) {  
23     this.result = result;  
24     latch.countDown();  
25 }  
26  
27 // ...  
28 }
```

具体应用：Lion 中的实例可以看出，变化点主要是对执行任务（即获取 Value）的实现。

ConfigManagerImpl 实现：

^ 代码块

Java



```
1 public class ConfigManagerImpl {
2     // 任务表
3     private ConcurrentMap<String, ConfigFuture> configFutures = new ConcurrentHashMap<String, ConfigFuture>(
4
5     protected String doGet(String key, String defaultValue, ConfigContext context) {
6         // ...
7         ConfigFuture<ConfigValue> existingFuture = null;
8         ConfigFuture future = new ConfigFuture(key);
9         existingFuture = configFutures.putIfAbsent(key, future);
10        if (existingFuture != null) {
11            // 如果之前有任务在执行了, 那么其他线程阻塞等待获取结果
12            try {
13                // 阻塞等待获取结果
14                configValue = existingFuture.get(metaConfigManager.getPollTimeout(), TimeUnit.MILLISECONDS);
15            } catch (Exception e) {
16                // ... 异常处理
17            }
18        } else {
19            // 执行任务, 执行成功后通知其他线程获取配置
20            try {
21                // 执行任务: 加载本地/ 远程的配置 【此处可以替换为其他获取值的实现】
22                configValue = configLoaderManager.get(key, context);
23                // 塞值 并 通知其他线程
24                future.setResult(configValue);
25            } catch (Throwable e) {
26                future.setResult(e);
27                throw new LionException(e);
28            } finally {
29                // 删除任务
30                configFutures.remove(key);
31            }
32        }
33        // ...

```

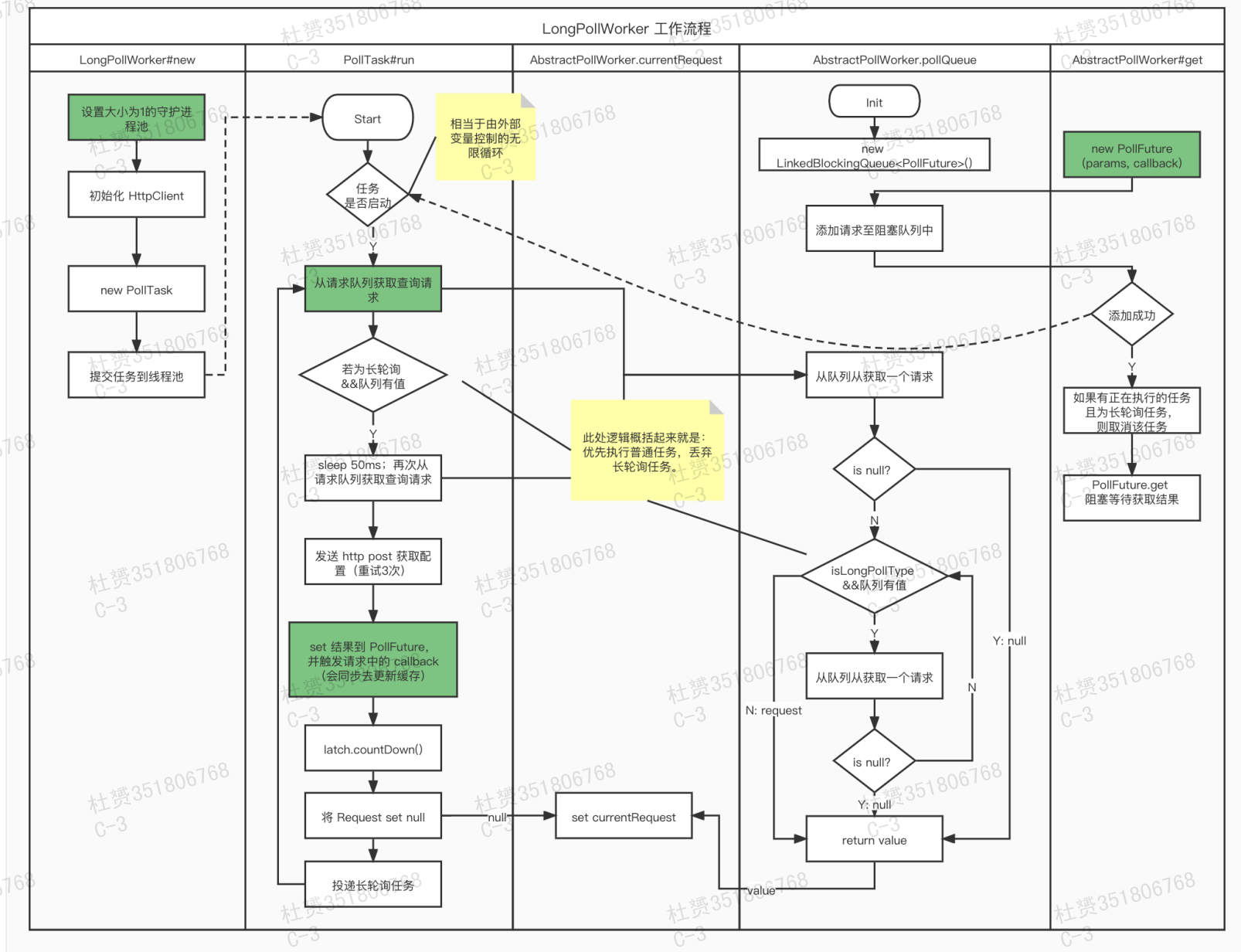
```
34  
35 }  
}
```

思考：ConfigFuture 中的 Key 字段用途是什么？从逻辑上看没有 key 也能正常跑。

2、HttpConfigLoader 中的 LongPollWorker：守护线程与阻塞队列实现循环任务处理

LongPollWorker 的作用：

- 实现任务处理框架不断执行投递的任务
- 任务的提交和优先执行：优先执行普通任务，其次执行长轮询任务。在系统空闲时（指无普通任务）自动投递长轮询任务。



- 其核心拆解为如下几个部分：
- 1、守护线程 `newFixedThreadPool`：设置固定大小为1的守护线程，并提交一个由外部共享变量控制的无限循环任务 Future
 - 2、阻塞队列 `LinkedBlockingQueue`：用于存储查询请求

上述两个部分形成了一个无限循环的执行框架

3、请求任务 PollFuture：存放了请求信息与回调方法，是驱动整个执行过程开始和结束的对象

Ps：对于长轮询请求而言，只有他是 50 ms 内唯一的请求才会被执行。

链路：获取请求->如果是长轮询请求则丢弃，直到他是最后一个请求->睡50ms->如果队列中又有新请求了，则拉取->如果是长轮询请求则丢弃，直到他是最后一个请求。

讨论：这个逻辑是不是用**优先队列**实现就能避免那么多特殊逻辑处理？

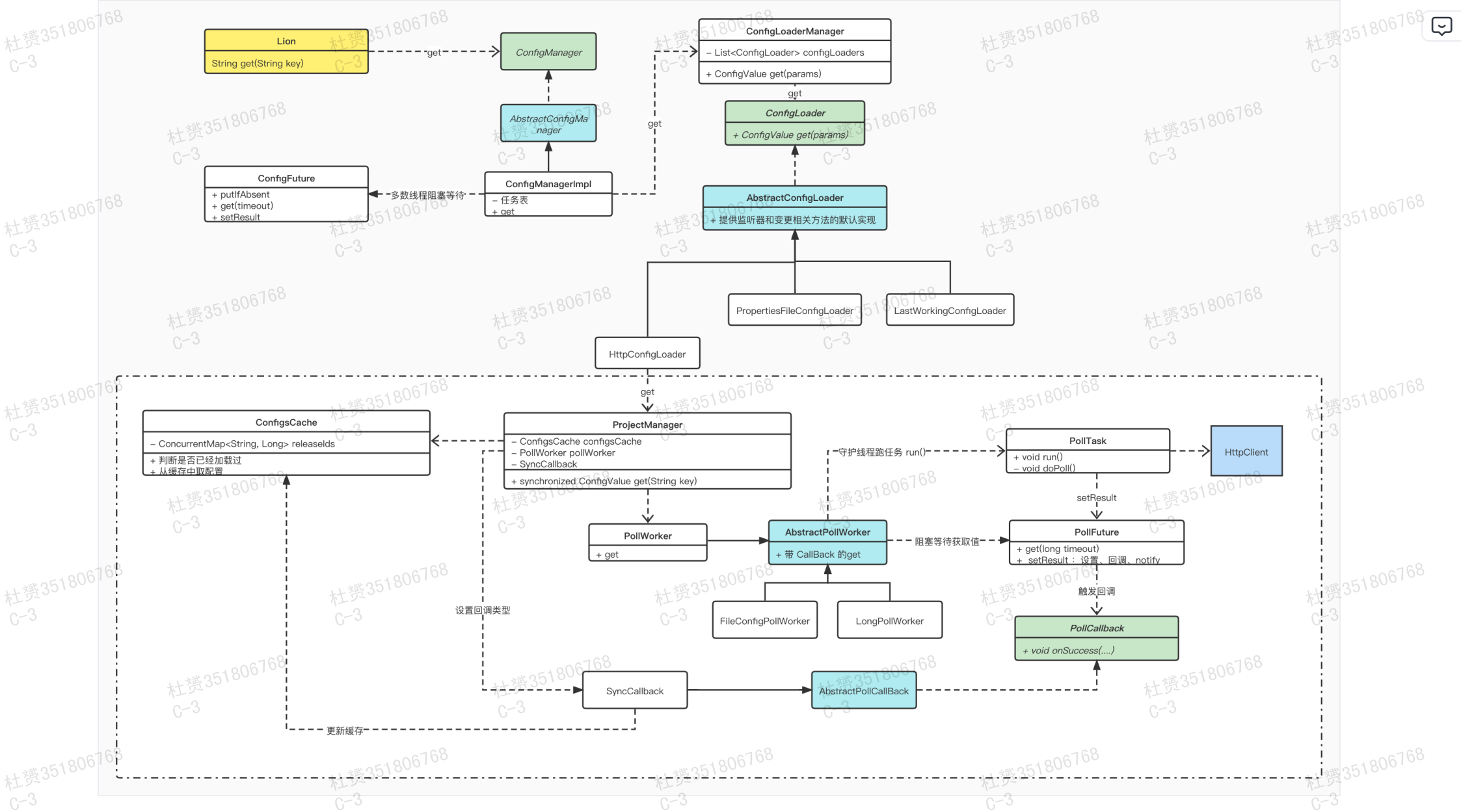
四、总结

4.1 一句话总结

Lion 在整个请求过程中做了较多性能和并发上的优化，使用内存 Cache 和 本地文件保障了读取时的性能和稳定性，CountDownLatch、阻塞队列、原子类、Sync 等并发组件保证并发时的数据读写安全。

4.2 过程类图

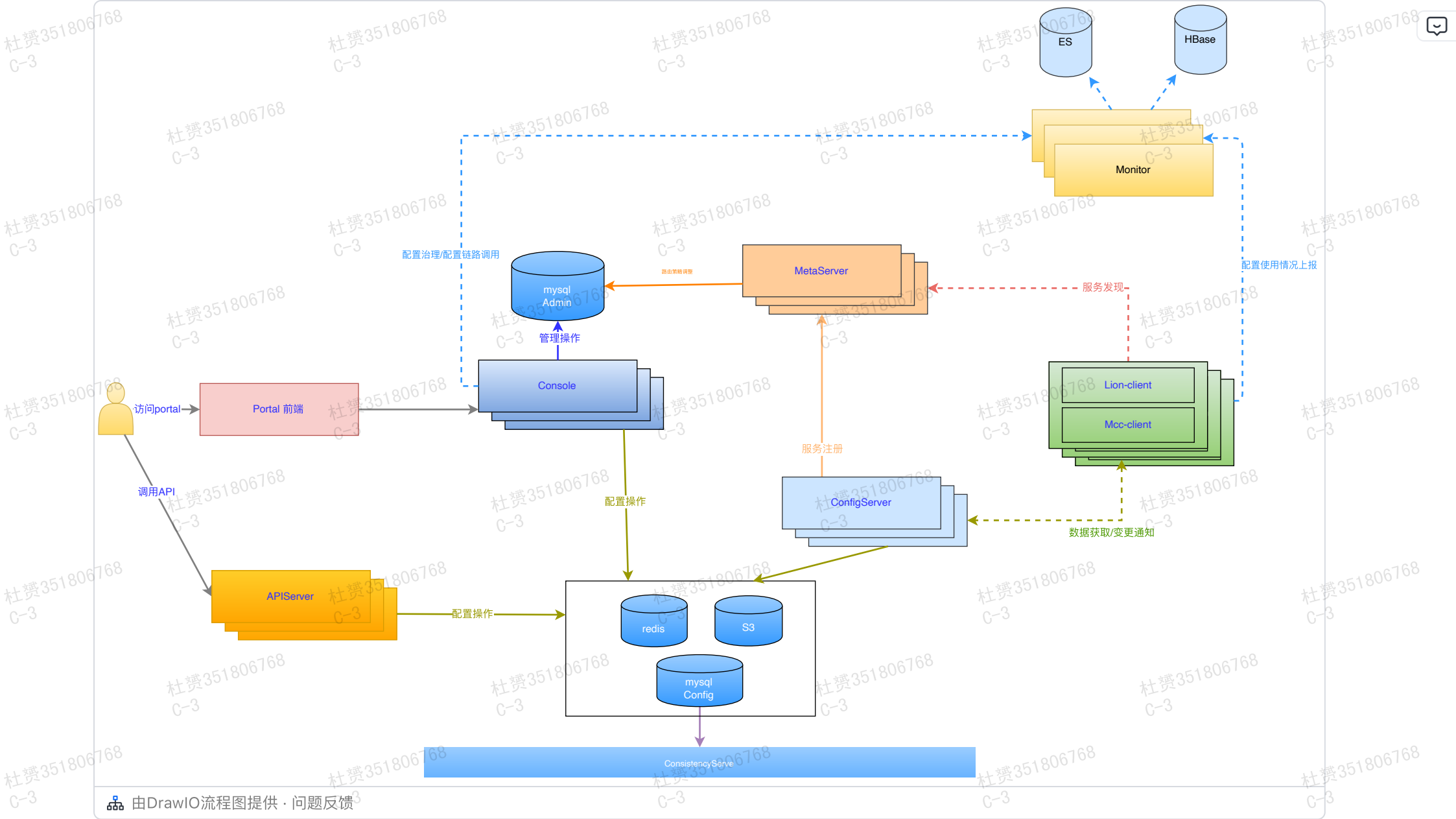
下附 Lion Get 配置过程中涉及到的类图，便于有个宏观认识：



架构图

Lion 架构

整体架构

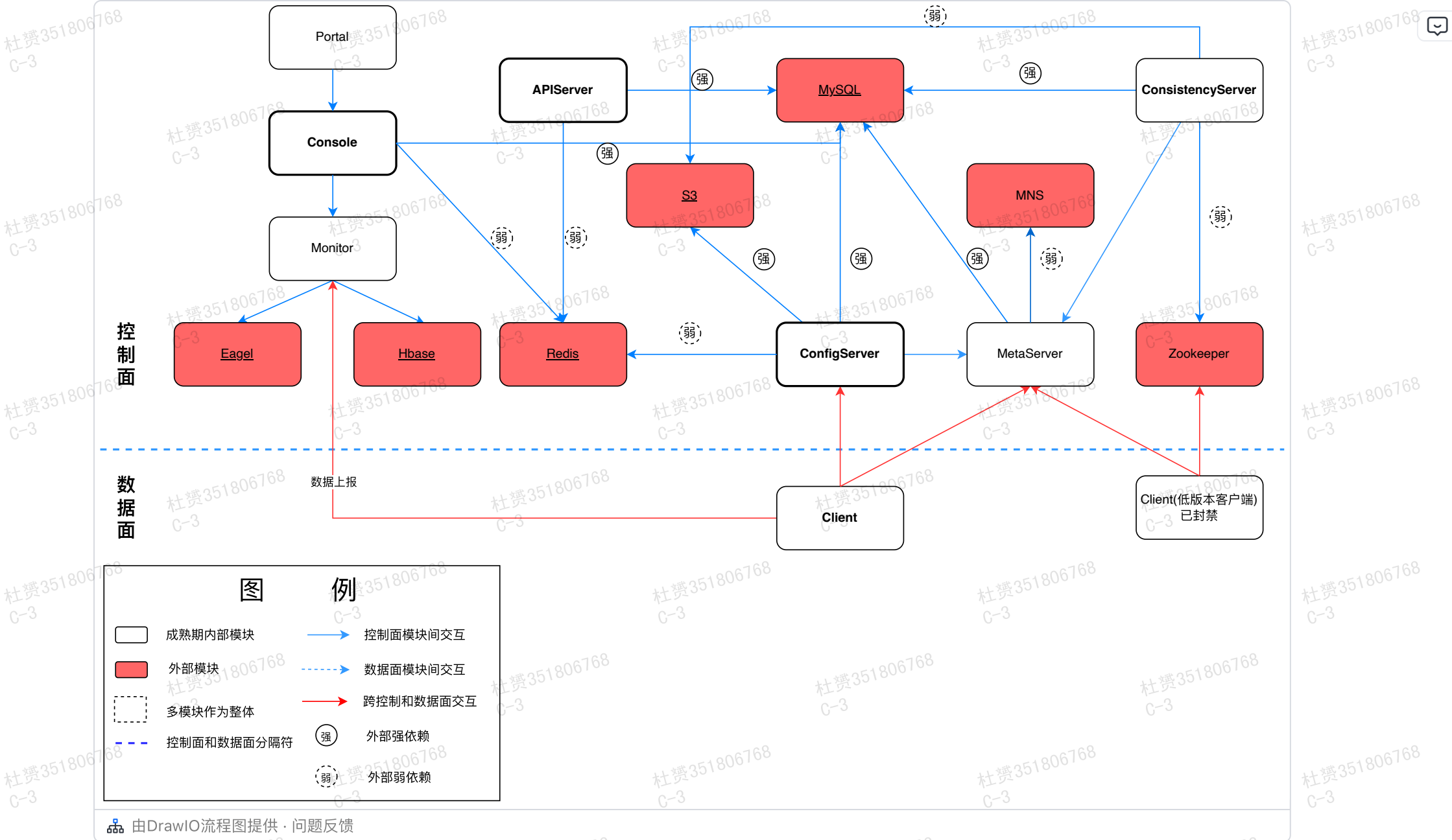


模块介绍

- **Portal**: 提供管理界面供用户进行配置管理。
- **Console**: 提供Portal调用及运维管理功能。
- **APIServer**: 提供配置相关接口，增加配置相关扩展能力。

ConfigServer：提供配置获取和配置推送能力，支持全量推送、灰度推送。

- Sidecar：为使用配置服务的应用提供服务接口，减少资源消耗，解决多语言问题，增强扩展能力。
- MetaServer：提供ConfigServer的服务注册发现功能。
- Lion-client|Mcc-client：配置获取客户端，当连接不上Sidecar时切换到配置服务器上。
- ConsistencyServer：同步模块，为实现两地读写多活，保证相同配置最终一致性。
- Monitor：收集客户端使用数据，配置变更推送情况。



- Portal：提供管理界面供用户进行配置管理。
- Console：提供Portal调用及运维管理功能。
- APIServer：提供配置相关接口，增加配置相关扩展能力。
- ConfigServer：Lion配置服务模块，服务客户端，提供配置获取和配置推送能力，支持全量推送、灰度推送。

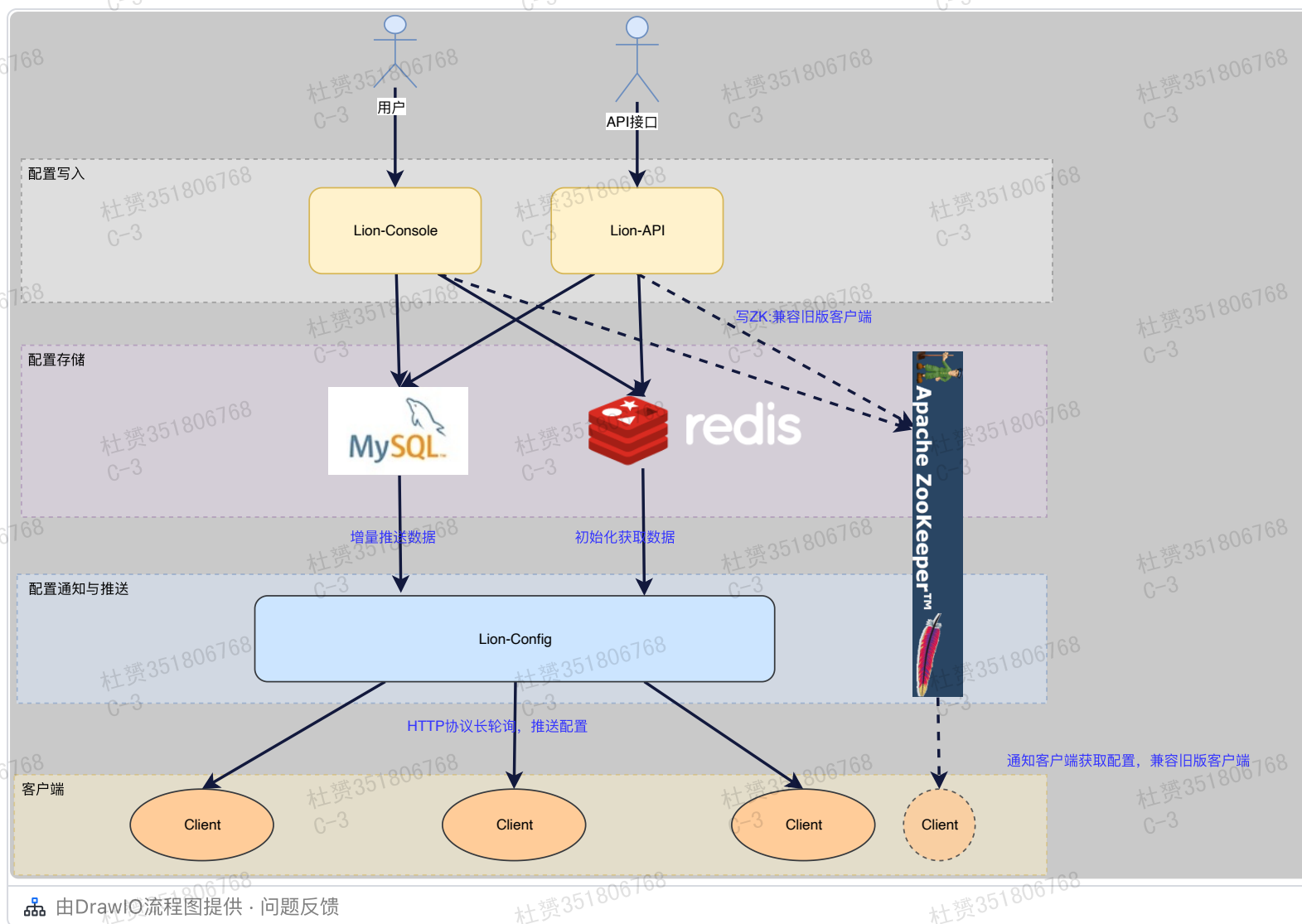
MetaServer：提供ConfigServer的服务注册发现功能；为客户端提供基础元数据。

- Lion-client/Mcc-client：客户端，为应用提供配置获取、配置实时更新等功能。
- ConsistencyServer：同步模块，为实现两地读写多活，保证相同配置最终一致性。
- Monitor：收集客户端使用数据、配置变更推送情况，用于配置治理以及配置推送链路。
- Scanner：扫描探测ConfigServer、ConsistencyServer模块可用情况。
- MGR：Mysql MGR集群，分为admin和lion两个库，admin保存管理数据，lion保存配置数据。

Lion采用异地多活架构，北京 & 上海各部署一套数据中心，支持双写，通过自研同步组件（Consistency）保证两个数据中心间数据一致性。业务配置存储主要使用 MySQL, Redis 作为存储介质（ZK 即将下线）。ps怎么保证数据一致性？

模块名称	主要功能	备注
lion-config	用于 client 获取配置与监听配置变更	- client 与 config 使用长轮询通信 - config 集群北上多机房部署，支持多集群
lion-consistency	- 不同数据中心间数据同步 - 同一数据中心，不同存储介质数据同步： - mysql -> redis - mysql -> zookeeper	- 北上双集群部署 - 不同集群间用于数据中心数据同步 - 同一集群内用于不同存储介质数据同步
lion-meta	- 实现 config, consistency 服务发现功能 - consistency 同步数据分区策略控制 - 用于 client 获取相关元数据：config 节点列表，黑白名单项目等	弱依赖 mns 实现节点注册发现功能，在 mns 不可测策略
lion-client	与 config 服务交互实现配置获取/监听功能	- 定时从 meta 获取可用 config 节点列表，并通过客户端 - config 服务不可用时，从本地文件获取配置（可能过期） - 支持多语言 sdk: java, node, go, c++

lion-api	提供 http open api：用于业务方通过 http 实现配置的变更与查询
lion-console	管理端交互，提供配置增删改查功能



旧版MCC使用Zookeeper作为底层，可能由于ZK的Znode节点有1M的大小限制就该为了Redis+ MySQL DB的形式新版Lion进行了架构重构，主要变化包括：

1. 存储层改造：



- 主存储从ZooKeeper迁移到关系型数据库（MySQL）
- 引入分布式缓存（Redis）提升读性能
- ZooKeeper仅用于元数据和协调，不再存储具体配置

2. 服务层改造：

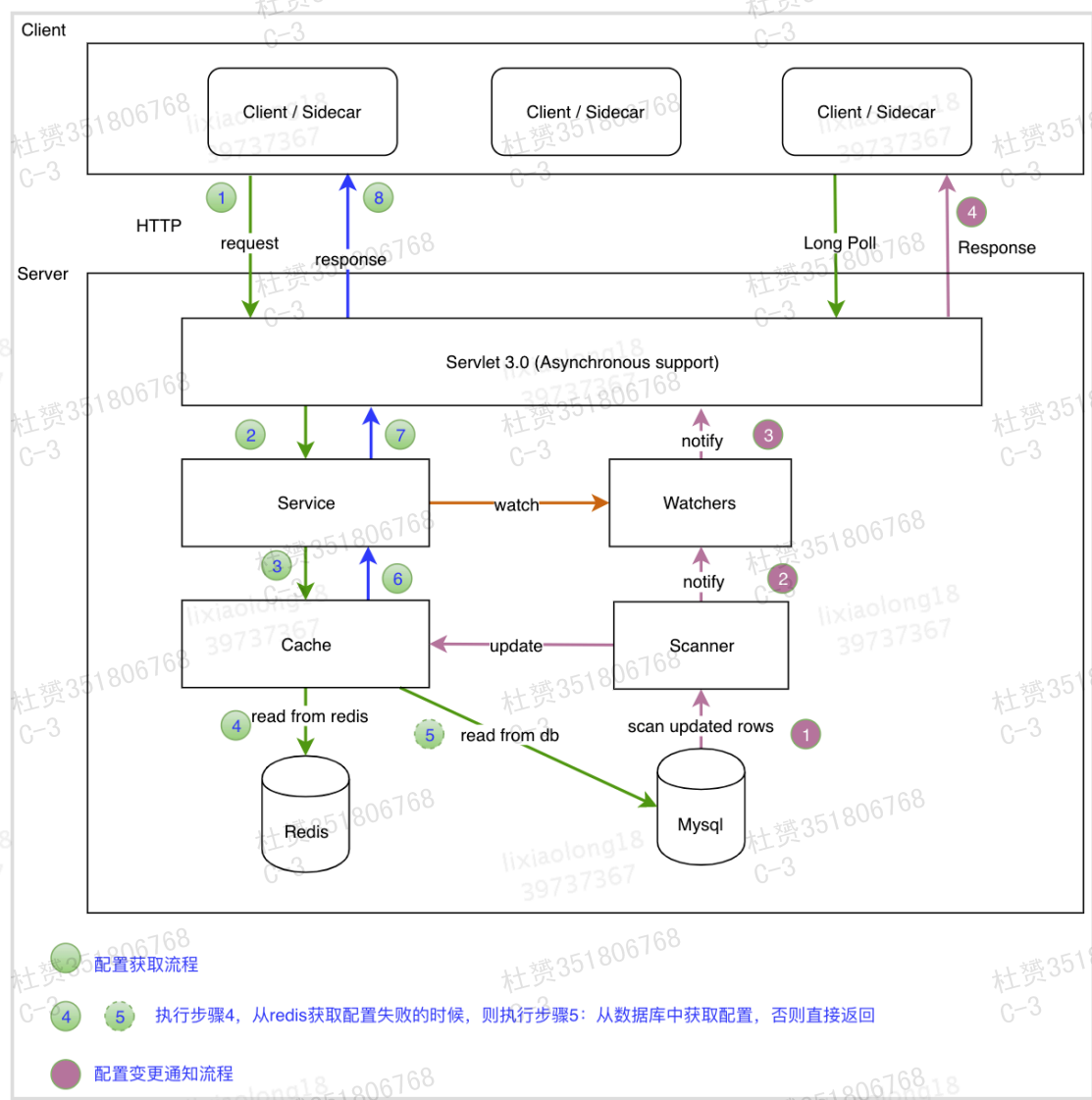
- 采用微服务架构，拆分为管理服务、配置服务等
- 引入消息队列（Kafka）实现配置变更通知
- 实现多级缓存机制，减轻存储层压力

3. 客户端改造：

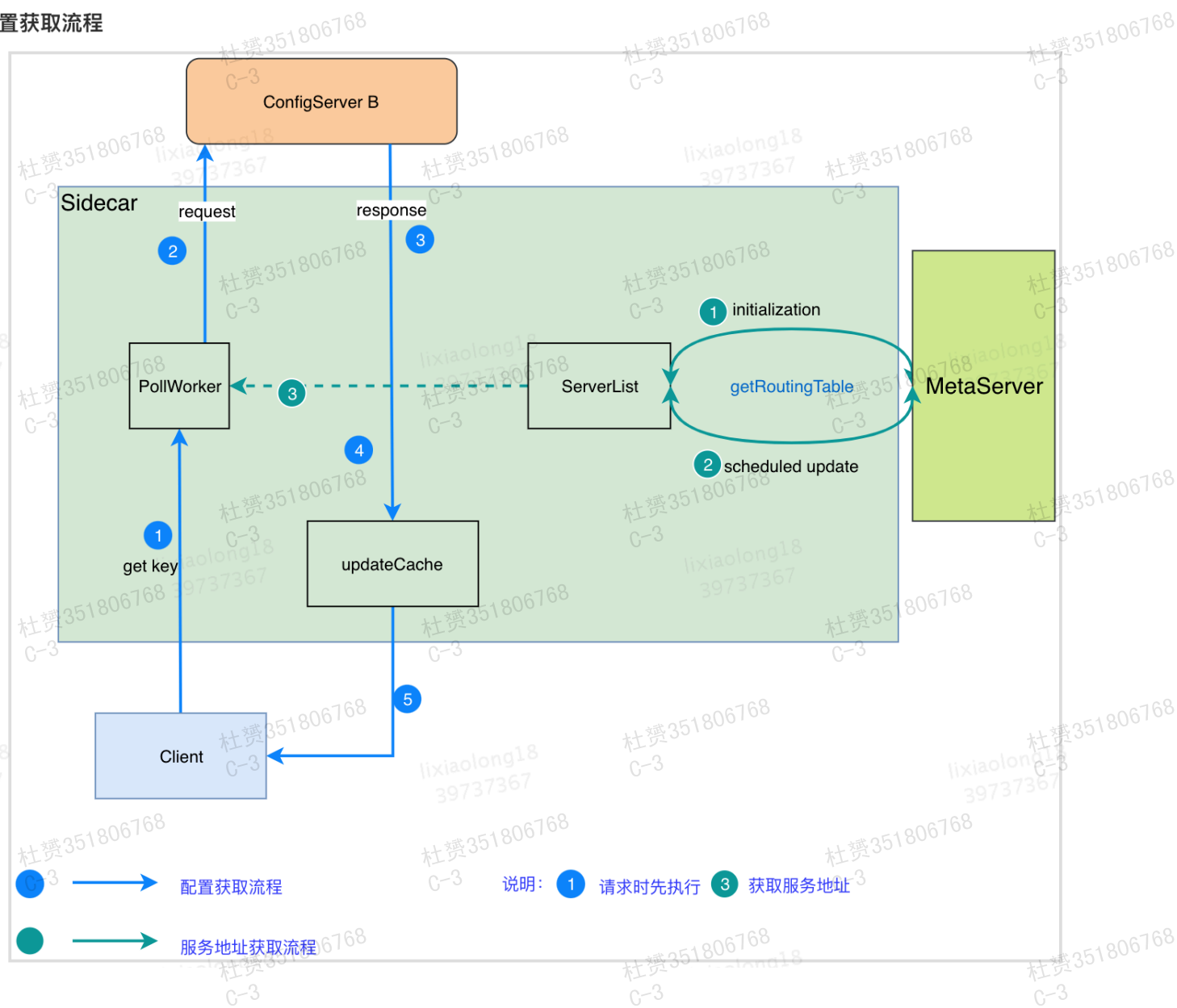
- 增强本地缓存能力
- 支持配置灰度发布
- 提供更丰富的降级策略

数据下发和更新模块架构

配置获取&推送:



配置获取流程

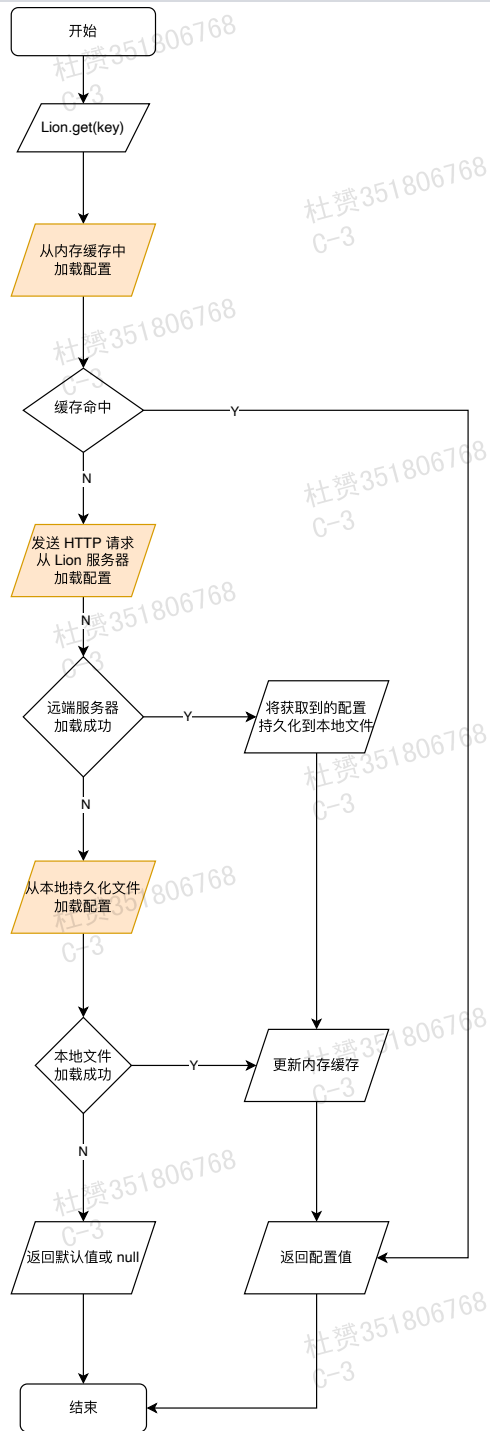


核心功能

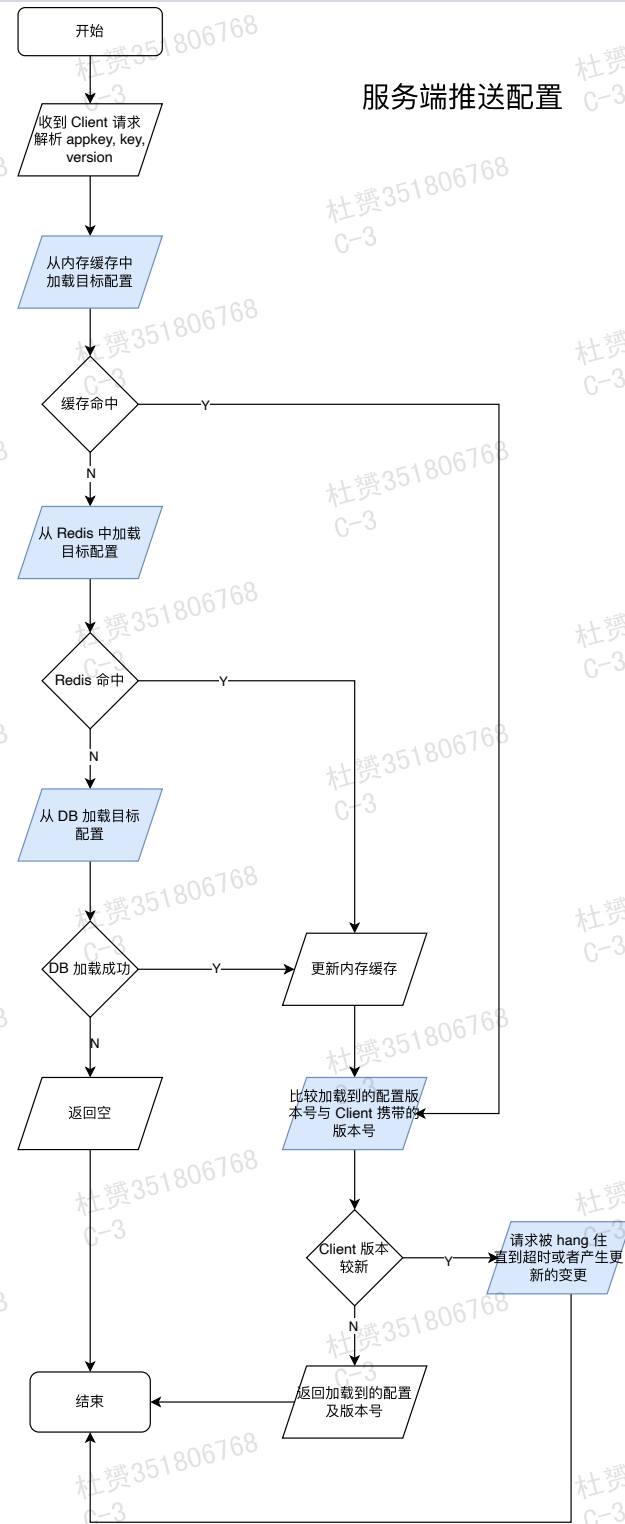
配置获取与推送

客户端获取配置缺少了点东西，具体可以见 `get` 源码分析

客户端获取配置



服务端推送配置



客户端获取配置：

1. 加载策略 外部文件、内存缓存、环境变量系统变量、Lion远程服务端、本地持久化文件
2. 在从服务端某个节点加载失败时，会及时切换到其他可用节点，直到超时；之后会**降级到本地文件（可能仍是上次获取到的旧值）**
3. 一个问题：如果大量的请求同时需要访问Lion远程服务器，会导致服务崩溃吗？怎么解决的？ 见**get源码分析**

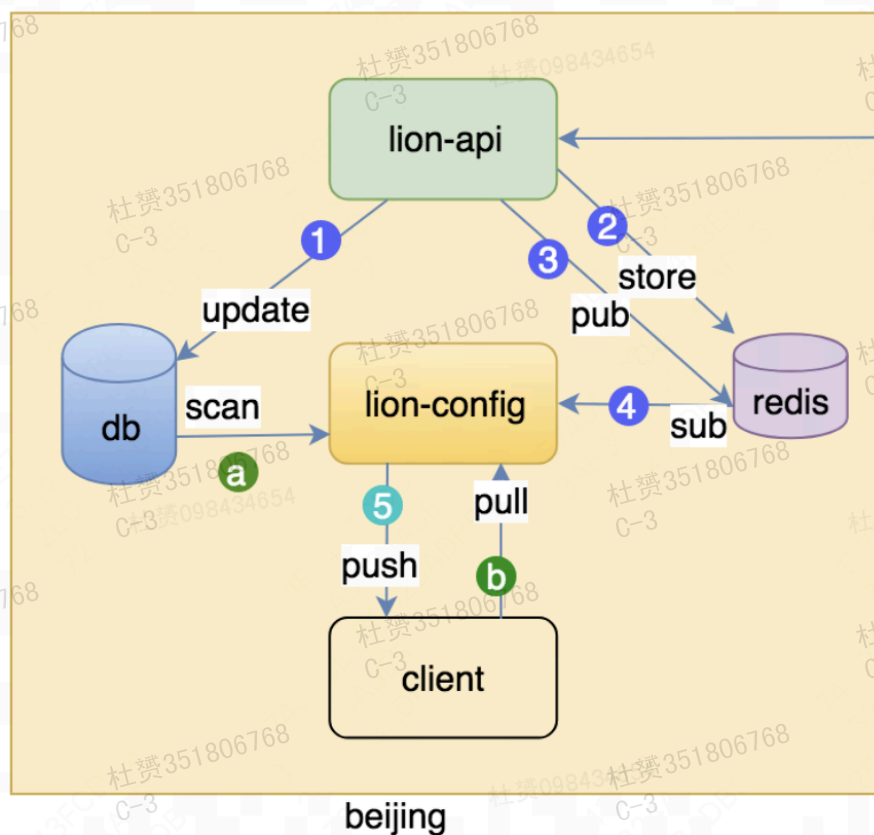
服务端处理请求：

1. 服务端收到客户端的获取请求后，也就是对应了访问Lion远程服务端，服务端依次从内存缓存、redis、db加载配置
2. 配置加载成功之后，需要比较 Client 携带的配置版本号（c_version）与服务端加载到的配置版本号（s_version）：
 - a. 如果 $c_version == 0$ ，则说明 Client 首次获取该配置，此时直接返回服务端配置
 - b. 如果 $c_version != 0$ 且 $c_version < s_version$ ，则说明该配置在 Client 上次加载之后又更新过，此时需要将最新配置返回
 - c. 如果 $c_version != 0$ 且 $c_version \geq s_version$ ，则说明 Client 本地配置已经是最新值，此时请求被 hang 住，直到该配置被更新或者请求超时（100s）

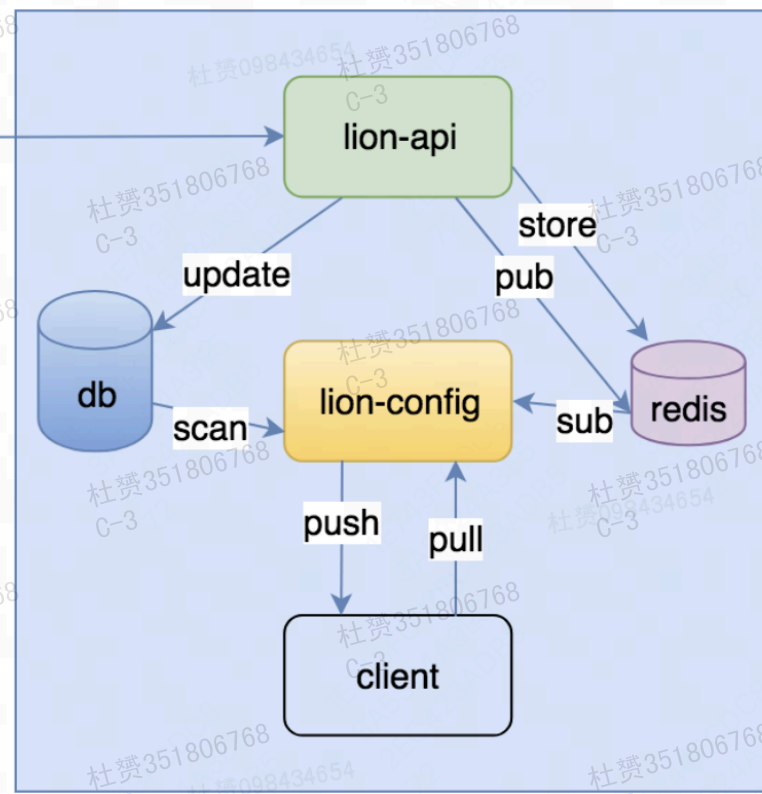
配置同步--服务端

一致性保障

北京和上海有两个数据中心，每个数据中心分别有 DB, Redis, ZK(即将下线)。为了实现异地多活，需要保证北上数据的一致性。



beijing



shanghai

- ① 更新数据库
- ② 更新redis
- ③ redis pubsub:publish
- ④ redis pubsub:onmessage
- ⑤ 推送配置，接收来自4和a的配置变更
- a 定时扫表获取最新更新
- b 客户端定时拉取全量数据
- c 地区间数据同步

Lion是最终一致性，通过Consistency保障，整体可以分为：不同数据中心的一致性、同一数据中心不同存储介质的一致性

ps怎么做的？需要补充 具体做法如下

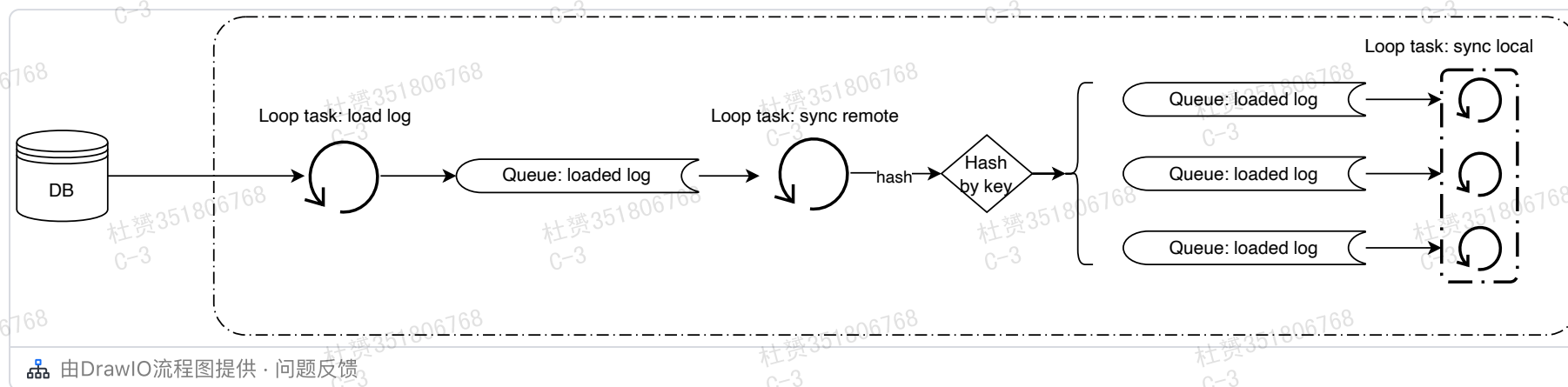
同步模型

为了实现数据最终一致性，lion 对每次配置变更都会记录一条 log，被称为同步日志（sync log）。该 log 模型基本如下：

key	release_key	local_db_status	remote_db_status	zk_status	redis_status	status
配置唯一标识	配置变更唯一标识	同侧 db 状态: 0 未同步, 1 同步成功	异地 db 状态: 0 未同步, 1 同步成功	同侧 zk 状态: 0 未同步, 1 同步成功	同侧 redis 状态: 0 未同步, 1 同步成功	本次变更 0 未加载,

不管是通过 http api 接口还是管理端变更配置，只会更新同侧的 db，并不会同步更新同侧数据中心内的 zk, redis（提高接口性能）

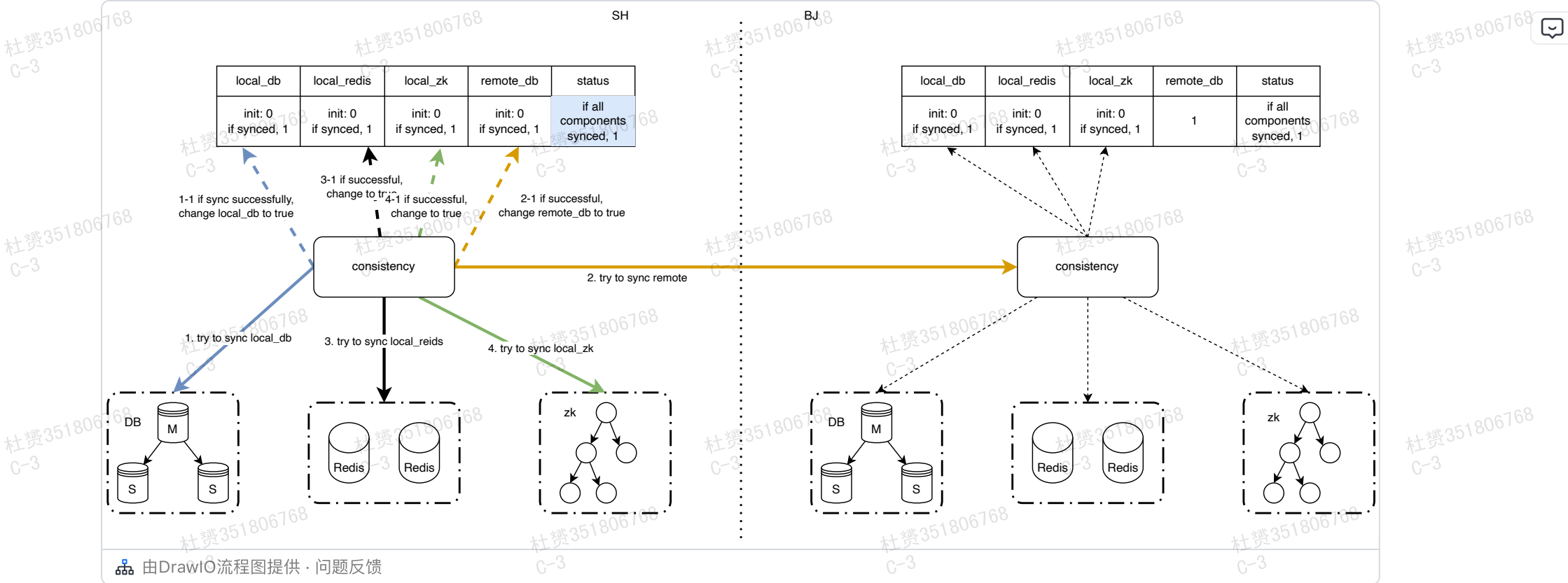
同步流程



同步流程的步骤：

- log 加载：** 将未同步完成的日志从 db 中加载到内存队列中
- 异地同步：** 优先将 log 批量同步到异地；异地同步成功后将 log hash 到内存队列中
- 本地同步：** 将 log 顺序同步到本地 db, redis, zk 中

详细流程图



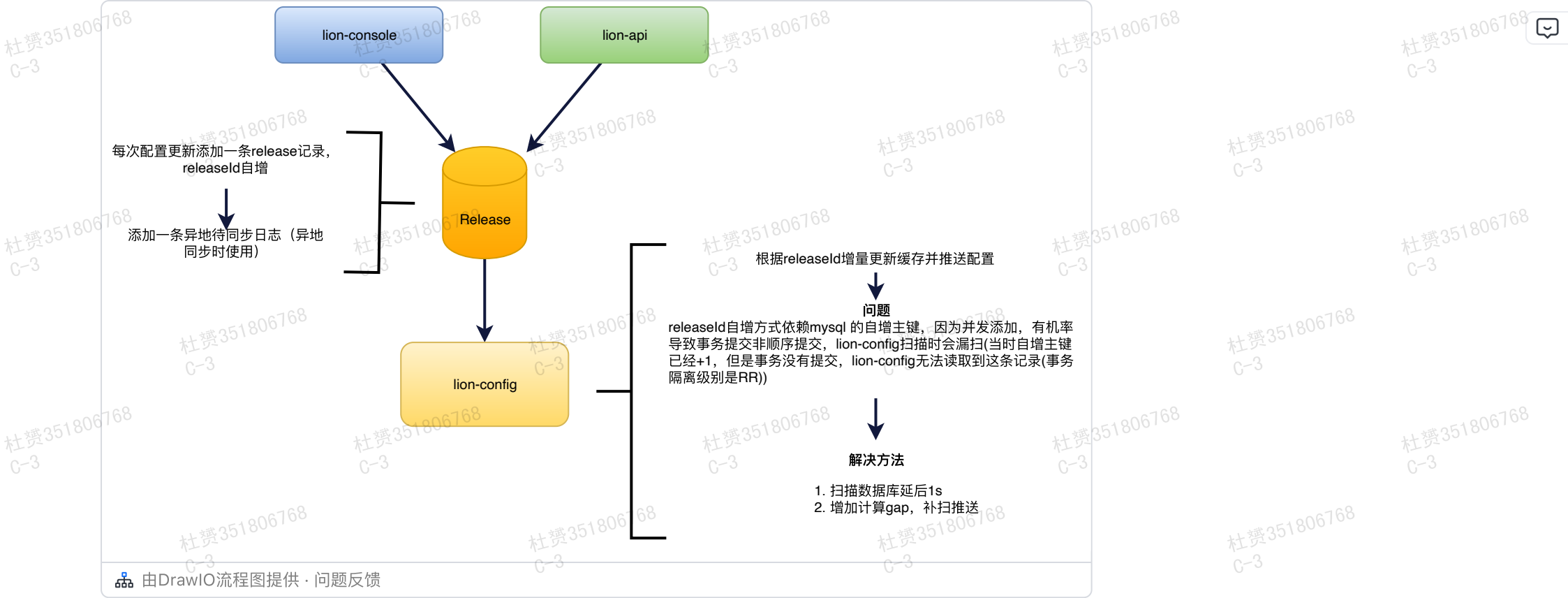
结合 log 中各个同步状态进一步分析:

- 本地 db 同步**: 如果本地 db 未同步, 则首先同步 db。如果同步成功, 则 $\text{log.local_db} = 1$ 。(对 log 产生侧来说, local_db 默认 1, 即默认不同步)
- 异地同步**: 该流程为异步操作, 仅仅是将 log 发送到异地并被接收, 并在稍后将配置回写到 db, redis, zk。如果同步成功, 则 $\text{log.remote_db} = 1$ 。
- 本地 redis 同步**: 如果同步成功, 则 $\text{log.local_redis} = 1$ 。
- 本地 zk 同步**: 如果同步成功, 则 $\text{log.local_zk} = 1$ 。

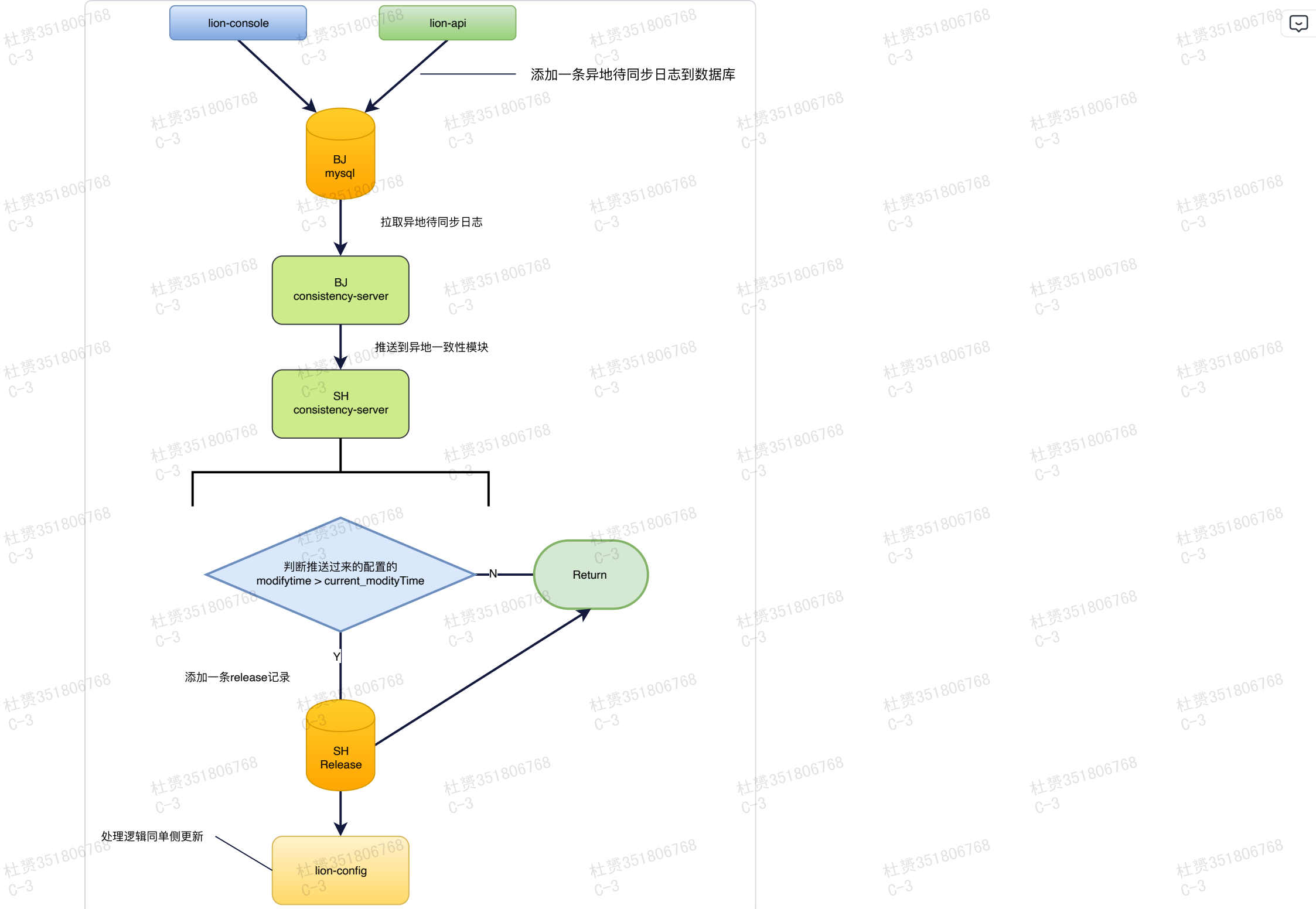
北上数据同步时, 如何保证同一条 log 不被重复同步?

- log 通过 status 区分同步状态; 同步任务只会加载 $\text{status} = 0$ 的记录, 确保同一条 log 不会被重复加载
- log 被加载后会进行分区, 保证不同 consistency 节点不会同步相同的 log

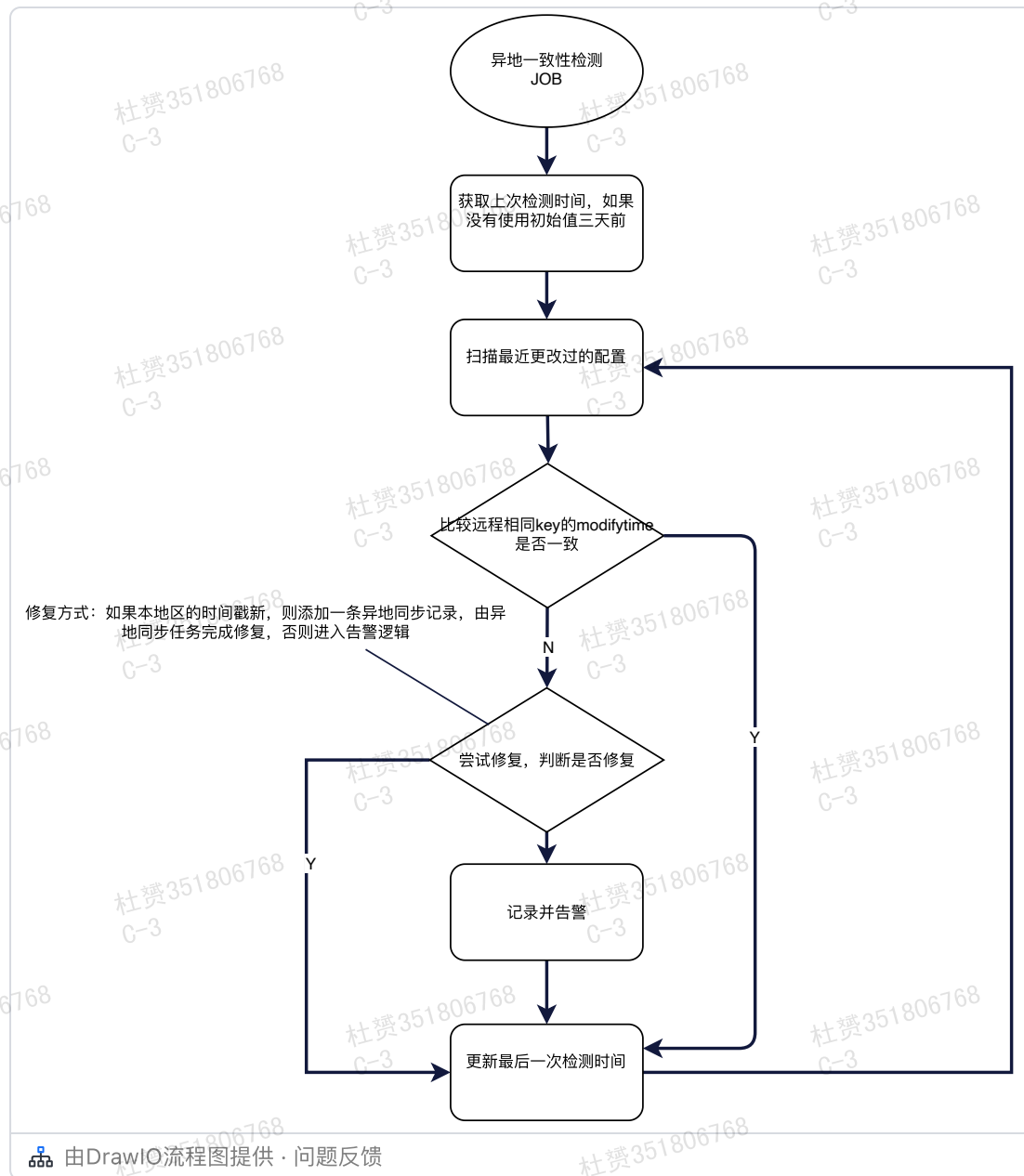
单侧数据更新



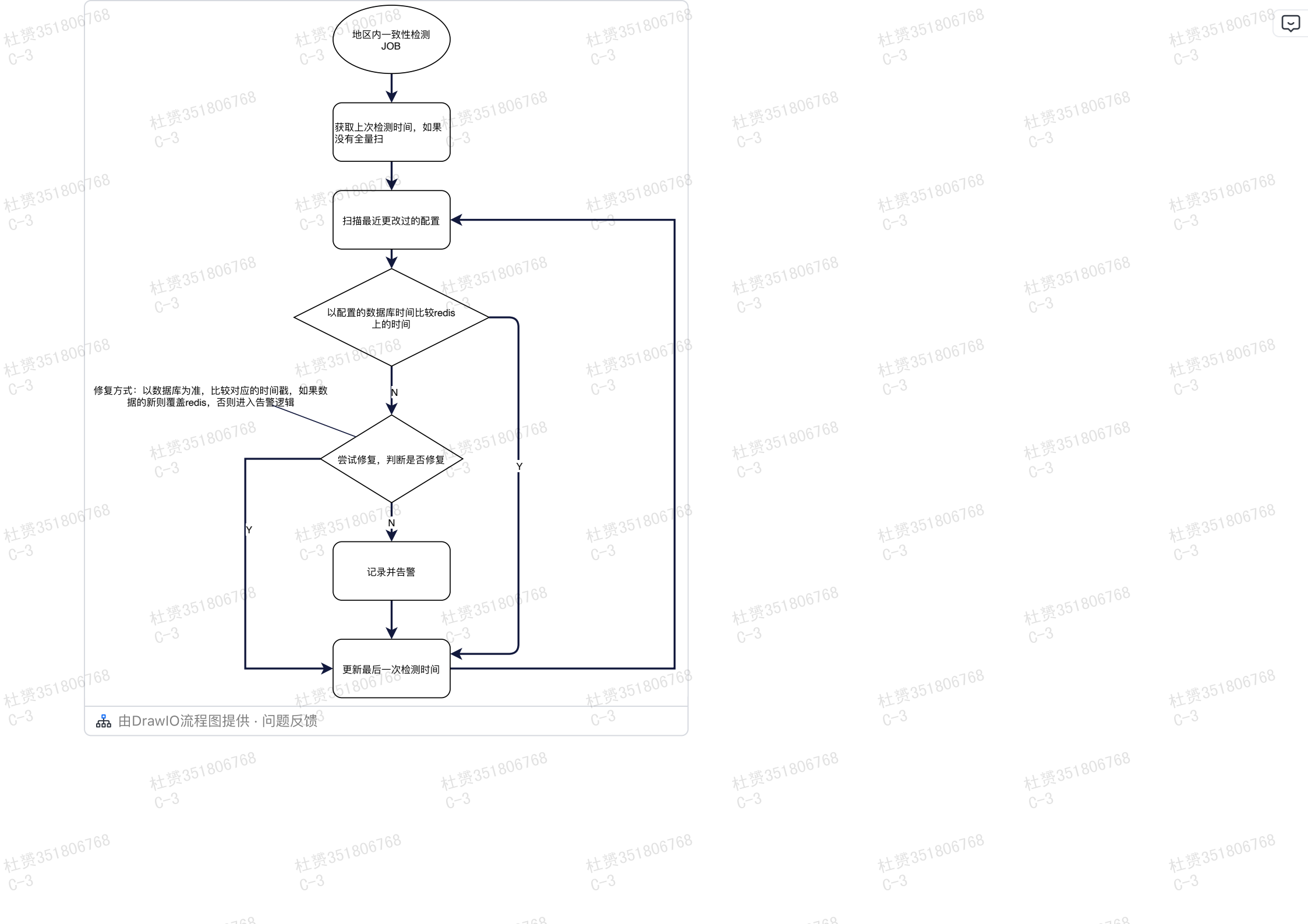
数据变更异步同步

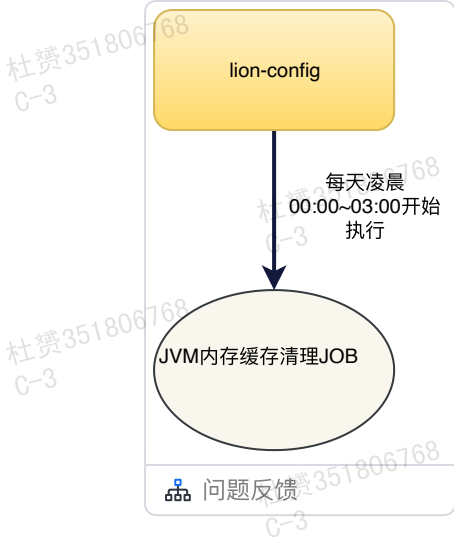


异地数据一致性保障



地区内redis和内存缓存一致

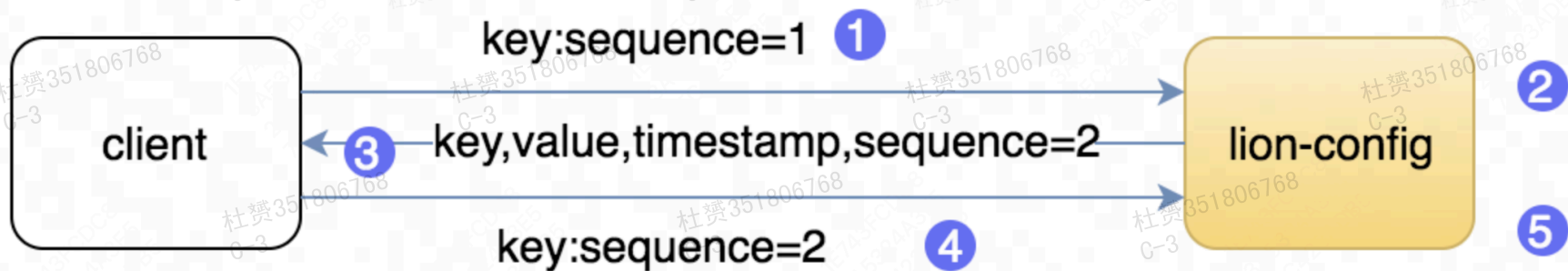




配置同步--客户端

服务端推送

增量推送



1

client 传送key,和sequence=1

2

判断sequence 1 < 2 , 增量推送最新配置

3

推送key,value,modifyTime,sequence=2到客户端

4

client 传送key,sequence=2

5

判断sequence 2 <= 2 , hold

客户端拉取

客户端定期(一小时)全量拉取配置

同步问题

1、北上数据同步时，如何保证同一条log不被重复同步？

- log 通过 status 区分同步状态；同步任务只会加载 status = 0 的记录，确保同一条 log 不会被重复加载
- log 被加载后会进行分区，保证不同 consistency 节点不会同步相同的 log

2、如何保持客户端服务端一致？

- 增量同步：客户端通过长轮询周期性地与服务端通信，请求信息携带了当前客户端配置的最新版本号，服务端通过比较版本号来判断客户端的配置是否过期，如果过期则更新客户端
- 全量同步：客户端定时全量同步本地配置，此时直接从服务端获取全量配置并更新本地过期配置

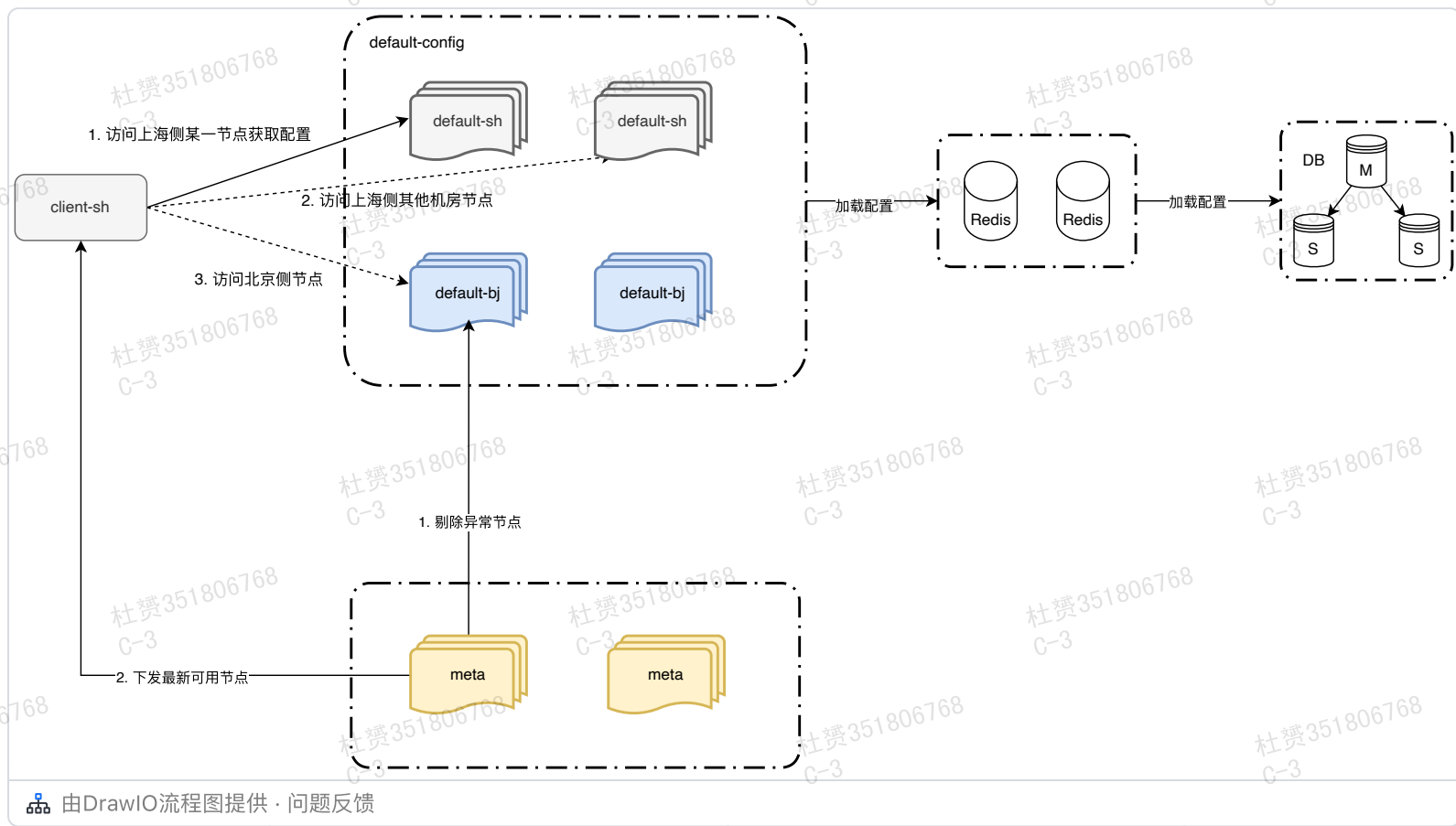
3、如何保证配置变更一定可以同步成功？

有三种同步机制：1. log 创建后被立即同步；2. 同步失败后会进行有补扫任务进行再次同步（50次）；3. 北上定时检测机制，对于不一致的数据会重新触发同步流程。

同步延迟5s之内！

可用性

config 服务北上两地多机房部署，支持弹性伸缩：



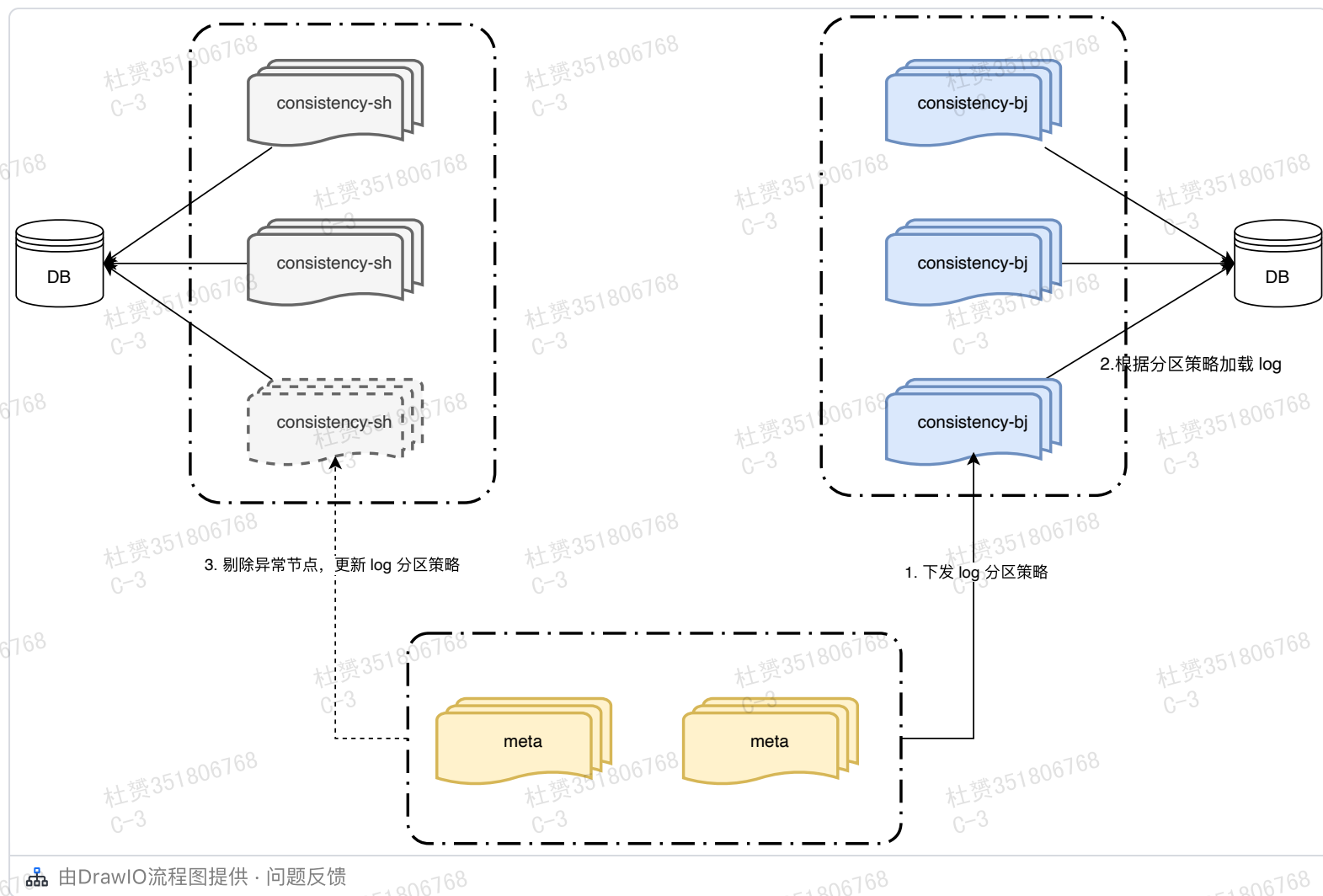
• 对于异常 config 节点，meta 会及时剔除，保证下发给 client 节点列表的可用性

- client 在请求 config 服务端获取配置时，优先访问同地域节点；同地域无可节点时，访问异地节点

如果所有 config 节点均不可用，则降级到本地持久化文件

config 服务加载配置时，强依赖 db，s3(文件配置)，弱依赖 redis：

- redis 不可用时，对配置获取 & 推送没有影响
- db，s3 不可用时，client 无法获取到最新配置（通过 config 内存缓存可以获取到旧配置）



consistency 服务北上两地多机房部署，支持弹性伸缩：

• 同步 log 加载强依赖 db，db 不可用时，同步任务无法执行

- consistency 节点异常时，meta 会及时剔除，并更新同步策略（尽可能降低节点变更对同步延迟的影响）

面

怎么实现统一管理实时推送？配置变更通知服务有哪些？分别的工作原理是什么？

怎么保证所有节点的最终一致性？

怎么保证高可用性的？如果所有的Lion config都挂了怎么办？

Lion保存配置又有redis db Lion-config缓存，怎么保证操作的原子性的？

Lion其他配置中心相比有什么区别？优势？

关于Lion的长连接怎么配置的？

超时设计

- 客户端发起长轮询请求后，服务端会保持连接直到以下条件之一满足：
 - 配置发生变更（立即响应）。
 - 达到**100秒超时**（服务端主动断开，客户端需重新发起请求）。

实时性保障

- 若配置在100秒内变更，服务端立即推送新值，客户端无需等待超时。
- 若超时无变更，客户端会重新发起长轮询，确保连接持续有效。

1. 统一管理实时推送与配置变更通知服务

Lion通过中心化数据管理和客户端/服务端架构实现统一管理与实时推送。配置变更通知服务包括：



• 监听器机制

- **全局监听**：监听所有配置变更
- **应用级监听**：针对特定应用的配置变更
- **配置项监听**：精确到单个配置项的变更
- **工作原理**：客户端通过长轮询（长连接）与服务端保持通信，服务端检测到配置变更后主动推送至客户端。长轮询
- **异步通知支持**
允许通过自定义线程池处理配置变更事件，避免阻塞主线程。

2. 保证所有节点的最终一致性

Lion采用**最终一致性模型**，具体机制如下：

- **多数据中心同步**
通过 **ConsistencyServer** 实现北京与上海数据中心的双向同步，确保异地数据最终一致。
- **同步流程**
 1. **日志记录**：每次配置变更生成同步日志（sync log），记录变更状态。
 2. **批量同步**：优先将日志批量同步至异地数据中心，再更新本地存储（DB→Redis→ZK）。
 3. **重试机制**：同步失败时触发补扫任务（最多50次），确保最终同步成功。
- **客户端一致性**
 - **增量推送**：服务端通过长轮询比较客户端与服务端版本号，推送差异配置。
 - **全量拉取**：客户端每小时全量拉取配置，修复潜在不一致。
 - **单调读一致**：单个客户端读取到新值后，不会回退到旧值。

3. 高可用性保障

- **服务冗余与降级**



- **ConfigServer多机房部署**：北上多机房部署，客户端优先访问同地域节点，异常时自动切换至异地节点。
- **本地缓存兜底**：若所有ConfigServer不可用，客户端降级至本地持久化文件（可能为旧值）。
- **存储容错**
 - **弱依赖Redis**：Redis不可用时，配置仍可从DB加载，不影响推送功能。
 - **DB/S3强依赖**：DB或S3故障时，客户端仅能获取旧配置（依赖内存缓存）。
- **健康检测**

MetaServer动态剔除异常节点，更新服务发现列表，确保客户端访问可用节点。

4. 若所有Lion Config服务宕机

1. **客户端降级**：从本地文件（`/data/appdatas/lion/lastWorkingConfig/`）读取最后一次生效的配置。
2. **功能影响**：
 - 无法获取新配置或监听变更，但应用可继续运行（依赖旧配置）。
 - 配置变更需等待服务恢复后同步。

Lion作为美团点评的分布式服务配置中心，在CAP理论中更倾向于CP（一致性和分区容错性）而非AP（可用性和分区容错性）。

Lion的CAP特性

为什么Lion是CP的：

1. **强一致性保证**：
 - Lion使用ZooKeeper作为底层存储和协调机制，而ZooKeeper本身就是一个CP系统
 - 配置更新时，Lion确保所有节点看到的配置数据是一致的，这对于配置中心至关重要
2. **分区容错性实现**：
 - 通过ZooKeeper的分布式协调能力，Lion能够在网络分区情况下继续工作



• 使用主从架构和选举机制确保在部分节点不可用时系统仍能正常运行

3. 可用性让步：

- 在极端网络分区情况下，Lion可能会牺牲部分可用性来保证数据一致性
- 当大多数ZooKeeper节点不可用时，Lion可能会拒绝配置更新操作，直到网络恢复

Lion如何保证CP特性

1. 基于ZooKeeper的一致性保证：

- 利用ZooKeeper的ZAB协议（Zookeeper Atomic Broadcast）实现数据一致性
- ZAB协议确保即使在leader节点故障的情况下，系统也能保持数据一致性

2. 配置数据的版本控制：

- Lion对配置数据实施严格的版本控制
- 客户端通过版本号检测确保获取到的是最新配置

3. 写操作的强一致性：

- 配置更新必须经过主节点，并同步到大多数节点后才返回成功
- 这确保了在网络分区恢复后，不会出现配置数据冲突

4. 读操作的一致性：

- 客户端可以从任何节点读取配置，但Lion确保读取到的是已经达成共识的数据
- 通过ZooKeeper的watch机制，客户端能够及时获取配置更新

5. 故障恢复机制：

- 当节点故障或网络分区发生时，Lion通过ZooKeeper的自动选举机制恢复服务
- 新的leader节点会确保系统状态的一致性，防止脑裂问题

为什么配置中心选择CP而非AP

对于配置中心这类系统，一致性通常比可用性更重要：

1. **错误配置的高风险**：不一致的配置可能导致系统行为异常，造成严重后果

2. **读多写少的特性**：配置更新相对较少，牺牲部分写可用性是可接受的

3. **业务影响范围**：配置错误可能影响整个系统，而短暂的配置不可用通常只影响配置更新操作

总结来说，Lion作为配置中心选择CP模型是合理的，它通过ZooKeeper的强一致性协议保证了配置数据在分布式环境中的一致性和可靠性，即使在网络分区的情况下也能保持数据正确性，这对于配置中心的核心功能至关重要。

Lion配置中心的CAP特性详解

CAP理论基础

CAP理论是分布式系统设计中的基础理论，提出在分布式系统中，一致性(Consistency)、可用性(Availability)和分区容错性(Partition tolerance)三者不可能同时满足：

- 一致性(C)**: 所有节点在同一时间看到的数据是一致的
- 可用性(A)**: 系统能够持续提供服务，每个请求都能得到响应
- 分区容错性(P)**: 系统在网络分区的情况下仍能继续运行

在实际的分布式系统中，由于网络分区是不可避免的，所以实际上只能在CA和CP之间做选择。

Lion作为CP系统的详细分析

1. 架构层面

Lion配置中心采用了多层架构设计：

- 存储层**: 基于ZooKeeper实现，负责配置数据的持久化和一致性保证
- 服务层**: 提供配置管理、版本控制、权限管理等功能
- 客户端层**: 提供配置获取、缓存和动态更新能力

这种架构使Lion能够在保证一致性的同时提供较高的可用性，但在极端情况下会优先保证一致性。

2. 一致性保证机制

ZooKeeper的ZAB协议

Lion底层依赖ZooKeeper的ZAB(ZooKeeper Atomic Broadcast)协议实现强一致性:

1. 两阶段提交:

- 第一阶段: Leader向所有Follower发送提案(Proposal)
- 第二阶段: 当超过半数Follower确认后, Leader发送提交命令

2. Leader选举:

- 当Leader节点失效时, 剩余节点会进行选举产生新Leader
- 新Leader会确保自己的数据是最新的, 防止数据不一致

3. 事务ID(zxid):

- 每个更新操作都有全局唯一的事务ID
- 保证了事务的全局有序性, 避免了数据冲突

配置版本控制

Lion实现了严格的配置版本控制机制:

1. 版本号管理:

- 每次配置更新都会生成新的版本号
- 客户端可以通过版本号检测配置是否最新

2. 原子更新:

- 配置更新是原子操作, 要么全部成功, 要么全部失败
- 防止配置数据处于中间状态

3. 事务日志:

- 记录所有配置变更操作

3. 分区容错性实现

集群部署

Lion通过多节点集群部署实现高可用：

1. 多数据中心部署：

- 跨机房、跨地域部署，提高物理隔离性
- 减少单点故障和区域性网络问题的影响

2. ZooKeeper集群：

- 通常部署奇数个节点(如3、5、7个)
- 只要多数节点($N/2+1$)正常，系统就能正常工作

网络分区处理

当发生网络分区时，Lion的处理策略：

1. 少数派分区：

- 无法形成多数派的分区将进入只读模式
- 拒绝配置更新操作，保证数据不会分叉

2. 多数派分区：

- 能够正常进行读写操作
- 选举新Leader继续提供服务

3. 分区恢复：

- 网络恢复后，少数派节点会自动同步数据
- 确保数据最终一致性

4. 可用性的权衡

虽然Lion优先保证一致性，但也采取了多种措施提高可用性：

客户端缓存机制

1. 本地缓存:

- 客户端会在本地缓存配置数据
- 即使配置中心暂时不可用，应用仍能使用缓存的配置

2. 定期刷新:

- 客户端定期从服务端拉取最新配置
- 减少对服务端的实时依赖

读写分离

1. 读操作优化:

- 读请求可以由任何节点处理
- 提高读操作的可用性和性能

2. 写操作集中:

- 写操作必须通过Leader节点
- 保证写操作的一致性

降级策略

1. 配置推送失败处理:

- 当配置推送失败时，客户端继续使用旧配置
- 防止因配置更新失败导致服务中断

2. 监控和告警:

- 实时监控配置中心状态

为什么配置中心选择CP而非AP的深入分析

1. 业务特性决定

配置中心的核心业务特性决定了它更适合CP模型：

1. 配置数据的关键性：

- 配置错误可能导致整个系统崩溃
- 错误的配置比没有配置更危险

2. 读写比例不均衡：

- 配置的读操作远多于写操作
- 牺牲写可用性对整体影响较小

3. 配置变更频率：

- 配置变更相对低频
- 短暂的写不可用是可接受的

2. 一致性与可用性的权衡

在不同场景下的权衡策略：

1. 正常运行时：

- 同时提供高一致性和高可用性
- 客户端缓存机制提供额外的可用性保障

2. 网络分区时：

- 优先保证一致性，可能牺牲部分可用性
- 少数派分区进入只读模式，防止数据分叉

3. 灾难恢复时：

- 通过数据同步机制恢复一致性

- 确保系统恢复后的数据正确性

3. 实际应用中的优化

美团在实际应用中对Lion进行了多方面优化：

1. 多级缓存：

- 服务端缓存减轻ZooKeeper压力
- 客户端缓存提高读性能和可用性

2. 批量操作优化：

- 合并多个配置更新操作
- 减少ZooKeeper写入次数，提高性能

3. 推拉结合的更新机制：

- 关键配置变更主动推送
- 非关键配置定期拉取
- 平衡实时性和系统负载

4. 分组管理：

- 按应用、环境、集群等维度分组管理配置
- 减少单个配置变更的影响范围

总结

Lion配置中心在CAP理论中选择了CP模型，通过ZooKeeper的强一致性协议保证配置数据的正确性和可靠性。虽然在极端情况下可能会牺牲部分可用性，但通过多层次的优化措施，Lion在实际应用中能够同时提供较高的一致性和可用性，满足美团点评这样大规模分布式系统的配置管理需求。

这种设计选择反映了配置中心这类基础服务的核心需求：配置数据的正确性比短暂的服务不可用更为重要，因为错误的配置可能导致整个系统的灾难性后果。

新架构下的CAP特性分析

尽管架构发生了变化，但Lion在CAP理论中的定位仍然偏向CP而非AP，只是实现方式发生了变化：

一致性(C)保证机制的变化

1. 数据库事务替代ZooKeeper事务：

- 使用MySQL的事务特性保证配置更新的原子性
- 通过数据库锁机制防止并发更新冲突

2. 多级一致性策略：

- 强一致性：关键配置通过同步方式确保即时一致
- 最终一致性：非关键配置允许短暂的不一致，通过异步机制最终达成一致

3. 版本控制增强：

- 更完善的配置版本管理机制
- 客户端支持配置版本检查和冲突处理

分区容错性(P)实现的变化

1. 多组件协同容错：

- 不再单一依赖ZooKeeper的分区容错能力
- 通过MySQL集群、Redis集群和消息队列共同提供分区容错能力

2. 服务网格化：

- 采用服务网格技术增强服务间通信的可靠性
- 支持跨区域部署和灾备切换

3. 多级降级策略：

- 定义更细粒度的故障场景和应对策略



可用性(A)提升策略

虽然仍然优先保证一致性，但新架构在可用性方面有显著提升：

1. 读写分离增强：

- 读操作完全从缓存层提供，不依赖主存储
- 写操作通过主从复制提高可用性

2. 多级缓存：

- 服务端缓存（Redis）
- 客户端缓存（内存）
- CDN边缘缓存（对部分静态配置）

3. 异地多活：

- 支持多区域部署
- 区域间数据同步机制
- 智能路由和负载均衡

新架构下的CAP权衡

新版Lion虽然减少了对ZooKeeper的依赖，但在CAP权衡上仍然偏向CP模型：

1. 一致性优先原则不变：

- 对于关键配置，仍然优先保证一致性
- 宁可拒绝服务也不提供错误配置

2. 可用性策略更灵活：

- 根据配置重要性分级，采用不同的一致性策略
- 非关键配置可以采用更宽松的一致性要求，提高可用性

3. 分区容错能力增强：

- 多组件协同提供更强的分区容错能力
- 更完善的故障检测和恢复机制

为什么仍然选择CP而非AP

即使在新架构下，Lion仍然选择CP而非AP的原因：

1. 配置中心的本质需求：

- 配置数据的正确性仍然是第一优先级
- 错误配置的危害远大于短暂不可用

2. 技术实现的进步：

- 新架构通过技术手段在保证CP的同时，显著提升了可用性
- 使得CP和A之间的权衡不再那么极端

3. 多级缓存的缓冲作用：

- 即使配置中心暂时不可用，应用仍能通过本地缓存正常运行
- 降低了对配置中心实时可用性的依赖

总结

新版Lion配置中心通过架构升级，减少了对ZooKeeper的依赖，采用更现代化的分布式系统设计。尽管实现方式发生了变化，但在CAP理论中的定位仍然偏向CP而非AP。

新架构通过多级缓存、读写分离、服务网格化等技术手段，在保证一致性的同时显著提升了可用性，使得系统在实际运行中能够同时提供较高的一致性和可用性，更好地满足美团这样大规模分布式系统的配置管理需求。

以下是关于 Lion 配置中心如何保证强一致性和实时推送 的详细技术解析，涵盖新旧架构的实现差异和核心机制：

一、强一致性的保证机制

1. 旧架构 (ZooKeeper 为主)

(1) ZAB 协议 (ZooKeeper Atomic Broadcast)

两阶段提交:

1. 提案阶段 (Proposal):

Leader 节点将配置变更作为提案广播给所有 Follower 节点。

2. 提交阶段 (Commit):

当超过半数 (N/2+1) Follower 确认后, Leader 发送提交命令, 所有节点应用变更。

特性:

全局有序性: 每个操作分配唯一事务 ID (zxid), 确保操作顺序一致。

Leader 选举: 故障时自动选举新 Leader, 新 Leader 必须拥有最新数据, 防止脑裂。

(2) 客户端一致性

版本号校验:

每个配置变更生成新版本号, 客户端通过版本号检测数据是否最新。

Watch 机制:

客户端注册监听 (Watch) 特定配置节点, 数据变更时 ZooKeeper 主动通知客户端。

2. 新架构 (MySQL + Redis + ZooKeeper)

(1) MySQL 事务保证

ACID 事务:

配置更新通过数据库事务 (如 `BEGIN TRANSACTION`) 保证原子性:

代码块

SQL

1 `BEGIN;`

2 `UPDATE config_table SET value = 'new_value' WHERE key = 'timeout';`

`INSERT INTO config_history (key, old_value, new_value) VALUES ('timeout', 'old_value', 'new_value');`

```
4 COMMIT;
```

```
5
```

- 行级锁:

防止并发更新冲突，确保同一时间只有一个事务能修改配置。

(2) 多级一致性策略

- 关键配置:

强一致性（同步更新）:

- 通过 ZooKeeper 或 MySQL 事务直接同步到所有节点。

- 非关键配置:

最终一致性（异步更新）:

- 通过消息队列（如 Kafka）广播变更，客户端定期拉取或被动推送。

(3) 版本控制与冲突解决

- 乐观锁:

客户端提交变更时携带版本号，服务端校验版本号是否匹配:

^ 代码块

```
1 UPDATE config_table SET value = 'new_value', version = version + 1
2 WHERE key = 'timeout' AND version = 5; // 版本号不匹配则失败
3
```

Java

- 冲突回滚:

版本冲突时，客户端重新拉取最新配置并重试。

二、实时推送的实现

1. 旧架构（ZooKeeper Watch 机制）

- 流程：

1. 客户端订阅配置节点的 Watch。
2. ZooKeeper 检测到节点变更后，主动通知所有监听该节点的客户端。
3. 客户端收到通知后，立即拉取最新配置。

- 特点：

- 低延迟：变更通知在毫秒级内推送。
- 一次性触发：Watch 触发后需重新注册。

2. 新架构（混合推送策略）

（1）关键配置：主动推送

- 长轮询（Long Polling）：

- 客户端发起请求后，服务端保持连接直到配置变更或超时。
- 变更时立即响应，客户端拉取最新数据。

- WebSocket 推送：

- 建立持久化连接，服务端主动推送变更事件。
- 适用于高频变更场景（如动态限流规则）。

（2）非关键配置：被动拉取

- 定期轮询：

- 客户端每隔一段时间（如 30 秒）主动拉取配置。
- 通过 `If-Modified-Since` 或 `ETag` 减少无效请求。

- 消息队列广播：

- 配置变更时，服务端发布事件到 Kafka。



■ 客户端订阅相关 Topic，异步接收变更通知。

(3) 多级缓存同步

1. 服务端缓存 (Redis):

- 配置变更时，先更新 MySQL，再同步到 Redis。
- 通过 Redis Pub/Sub 通知所有实例刷新缓存。

2. 客户端缓存:

- 收到推送后，客户端更新本地内存缓存。
- 未收到推送时，降级使用本地缓存（带版本校验）。

三、技术细节与优化

1. 强一致性优化

• 批量提交:

合并多个配置变更操作，减少 ZooKeeper/MySQL 的写入压力。

• 本地快照:

客户端缓存最新配置的快照，网络分区时仍可读取。

2. 实时推送优化

• 增量更新:

仅推送变更的配置项（而非全量），减少网络开销。

• 压缩传输:

使用 Protobuf 或 Snappy 压缩推送数据。

• 优先级队列:

关键配置的推送任务优先处理，确保低延迟。

3. 容错与降级



- **推送失败重试：**

采用指数退避策略重试（如 1s → 2s → 4s）。

- **客户端超时机制：**

推送超时后，客户端主动拉取最新配置。

四、总结

1. 强一致性保证

- **旧架构：**依赖 ZooKeeper 的 ZAB 协议，通过两阶段提交和版本号实现强一致。

- **新架构：**

- 关键配置：MySQL 事务 + ZooKeeper 同步。
- 非关键配置：消息队列异步 + 最终一致性。

2. 实时推送保证

- **旧架构：**ZooKeeper Watch 机制，低延迟但需重新注册。

- **新架构：**

- 关键配置：长轮询/ WebSocket 主动推送。
- 非关键配置：定期轮询 + 消息队列广播。
- 多级缓存同步：Redis Pub/Sub + 客户端本地缓存。

3. 核心思想

- **强一致性：**通过分布式协议（ZAB）、事务（MySQL）和版本控制，确保数据全局一致。
- **实时推送：**结合主动推送（长轮询/WebSocket）和被动拉取（轮询/消息队列），平衡实时性与系统负载。

这种设计使得 Lion 在复杂分布式环境中，既能保证配置数据的绝对正确性，又能满足业务对实时性的需求。

1. Lion 如何保证配置数据的最终一致性?

回答:

Lion 采用“服务端集中管理 + 客户端多级缓存”的架构实现最终一致性:

• 服务端:

- 配置变更通过 **ConsistencyServer** 模块 记录同步日志, 在多个数据中心间双向同步 (基于消息队列或数据库日志)。
- 使用 **版本号** 标记每次变更, 客户端通过版本校验避免脏读。

• 客户端:

- 长轮询机制**: 客户端与服务端保持长连接, 服务端检测到变更后主动推送最新配置 (5 秒内同步到消费者)。
- 本地缓存**: 配置首次加载后缓存在本地, 服务端不可用时降级使用缓存, 通过 **Last-Modified** 或 **ETag** 校验更新。
- 容错设计**:
 - 网络分区时, 客户端继续使用本地缓存, 分区恢复后通过版本号自动同步最新配置。
 - 通过 **Cat 监控** 跟踪同步延迟, 异常时触发告警。

下面也是

• 最终一致性实现:

- 同步日志 (Sync Log)**:
每次配置变更会生成一条同步日志 (记录变更状态、版本号、时间戳), 通过 **ConsistencyServer** 在异地数据中心间双向同步。
 - 同步流程**:
 - 变更写入本地 MySQL → 生成 Sync Log → 批量推送至异地数据中心。
 - 异地数据中心接收后, 先写入本地 MySQL, 再更新 Redis/ZooKeeper (若存在)。
 - 冲突解决**:
通过 **版本号 (Timestamp/Version)** 和 **Last-Write-Wins** 策略解决冲突 (后写入的覆盖先写入的)。
- 客户端同步**:

- **增量推送**：服务端通过长轮询比较客户端版本号，仅推送差异配置。
- **全量拉取**：客户端每小时全量拉取配置，修复潜在不一致（如网络分区导致漏推）。
- **单调读一致性**：单个客户端一旦读取到新值，后续读取不会回退到旧值（通过本地缓存版本号校验实现）。

2. Lion 的实时推送机制是如何实现的？

回答：

Lion 的实时推送依赖 **长轮询（Long Polling）** 和 **多级缓存协同**：

- **长轮询（Long Polling）**：
 - **服务端逻辑**：
 1. 客户端发起请求时，服务端检查配置版本号。
 2. 若版本一致，挂起请求（默认 30 秒超时）。
 3. 配置变更时，立即响应客户端并携带新版本数据。
 - **客户端逻辑**：
 - 收到响应后立即更新本地缓存，并重新发起长轮询。
 - 超时未响应时，降级为定期轮询（如 30 秒一次）。
- **多级缓存协同**：

缓存层级	作用	失效策略
服务端 Redis	减轻 MySQL 压力	配置变更时通过 Pub/Sub 通知刷新
客户端内存	快速读取，降级容灾	版本号校验 + TTL（如 5 分钟）
本地文件	持久化备份	启动时加载，变更时持久化

3. Lion 的异地多活架构如何设计？

回答：

Lion 的异地多活通过 **数据双向同步 + 一致性协议** 实现：

- **数据同步：**
 - **ConsistencyServer** 记录每次变更的同步日志，通过消息队列（如 Kafka）跨数据中心同步。
 - 采用 **最终一致性模型**，允许短暂延迟（通常 <5 秒）。
- **容灾设计：**
 - **多活数据中心：** 每个数据中心独立处理读写，通过全局版本号解决冲突。
 - **客户端路由：** 根据网络延迟自动选择最近的数据中心，故障时切换。
- **监控与恢复：**
 - 通过 **Cat 监控** 跟踪同步状态，异常时触发人工干预。
 - 数据冲突时，按时间戳或版本号回滚。

4. Lion 如何应对高并发读取场景？

回答：

Lion 通过 **多级缓存 + 读写分离** 优化高并发读取：

1. **客户端缓存：**
 - 配置首次加载后缓存在本地内存，后续请求直接读取，减少服务端压力。
 - 缓存失效策略：基于版本号或 TTL（如 5 分钟）。
2. **服务端缓存：**
 - 使用 Redis 缓存热点配置，降低数据库查询压力。
 - 通过 **Pub/Sub** 机制通知所有实例刷新缓存。
3. **读写分离：**

• 读操作从 Redis 或本地缓存获取，写操作异步同步到 MySQL/ZooKeeper。

• 通过 限流（Sentinel）防止突发流量击穿缓存。

5. Lion 的权限管理和审计如何实现？

回答：

Lion 的权限与审计基于 **RBAC 模型 + 操作日志**：

- **权限控制：**
 - 支持 **环境隔离**（开发/测试/生产），不同环境配置独立管理。
 - 细粒度权限（如“只读”“编辑”“发布”），通过 LDAP/SSO 集成企业账号体系。
- **操作审计：**
 - 所有配置变更记录 **操作日志**（谁、何时、修改了什么）。
 - 支持配置 **灰度发布** 和 **回滚**，通过版本号追溯历史变更。
 - 审计日志存储到 Elasticsearch，支持快速检索与告警。

6. Lion 的局限性是什么？

回答：

Lion 的局限性主要体现在以下场景：

1. **强一致性需求：**
 - 最终一致性模型不适用于对实时性要求极高的场景（如金融交易）。
2. **配置规模限制：**
 - 单个服务/环境的配置项总和有限（如 10MB），超大规模配置需拆分。
3. **网络分区影响：**
 - 极端网络分区时，客户端可能短暂使用旧配置，需业务容忍短暂不一致。

7. 对比 Apollo/ZooKeeper, Lion 的核心优势?

回答:

Lion 的优势在于 美团内部场景的深度优化:

- **实时性**: 长轮询 + 多级缓存, 比 Apollo 的定期轮询更实时。
- **容灾能力**: 异地多活设计, 比 ZooKeeper 的 CP 模型更适应高可用需求。
- **治理工具**: 集成 Cat 监控和配置治理, 便于追踪废弃配置。
- **成本**: 本地缓存降低对 ZooKeeper 的依赖, 减少运维复杂度。

Lion 如何保证 Redis/DB/Lion-Config 缓存的操作原子性?

原子性保障

- **写操作**:

1. **MySQL 事务**: 配置更新通过数据库事务 (ACID) 保证原子性。sql

```
BEGIN; UPDATE config_table SET value = 'new_value' WHERE key = 'timeout'; INSERT INTO config_history (key, old_value, new_value) VALUES ('timeout', 'old_value', 'new_value'); COMMIT;
```

2. **版本号校验**: 客户端提交变更时携带版本号, 服务端校验版本号是否匹配 (乐观锁)。

- **缓存同步**:

- **Redis Pub/Sub**: 配置变更时, 服务端通过发布订阅通知所有实例刷新缓存。
- **延迟双删**: 先删缓存 → 更新 DB → 再删缓存, 避免脏数据。

面试技巧

- **结合业务场景**: 例如, “在美团外卖的骑手调度系统中, Lion 的实时推送确保了动态限流规则的秒级生效。”

- **突出监控与容灾：**强调 Cat 监控、客户端降级、异地多活等设计。
- **对比其他方案：**如 ZooKeeper 的 CP 模型不适合高并发读，Apollo 的定期轮询延迟较高。

重点面试

Lion配置中心面试重点问题及详细回答

重点面试问题

以下是面试中最可能被问到的Lion配置中心相关问题，按重要性排序：

1. Lion配置中心的核心功能和架构

详细回答：

Lion是美团内部的配置管理平台，其核心功能包括：

1. **统一配置管理：**集中管理不同环境(dev/test/staging/prod)、不同Set泳道的配置，实现配置与代码分离。
2. **实时配置推送：**通过长轮询(Long Polling)机制，实现配置变更的秒级推送。当配置发生变更时，服务端会立即响应客户端的请求，客户端收到变更通知后更新本地缓存并触发监听器。
3. **版本管理与回滚：**所有配置变更都有版本记录，支持快速回滚到历史版本。
4. **灰度发布：**支持配置的灰度发布，可以指定部分客户端节点先接收新配置，观察一段时间后再全量发布。
5. **配置治理：**通过收集配置使用数据，帮助业务方了解配置使用情况，识别未使用的配置。

Lion的架构主要包括以下组件：

- **Portal：**提供Web管理界面，用于配置管理
- **Console：**提供Portal调用及运维管理功能
- **APIServer：**提供配置相关接口，增加配置扩展能力
- **ConfigServer：**服务客户端，提供配置获取和推送能力

• **MetaServer**: 提供ConfigServer的服务注册发现功能

- **客户端SDK**: 为应用提供配置获取、实时更新等功能
- **ConsistencyServer**: 同步模块, 保证配置最终一致性
- **Monitor**: 收集客户端使用数据和配置变更推送情况

2. Lion的一致性保证和配置推送机制

详细回答:

Lion提供的是最终一致性保证, 具体表现为:

1. **全局最终一致性**: 一定时间后(通常5秒内)所有客户端读到的配置值最终达到完全一致。
2. **单调读一致性**: 单个客户端角度, 当客户端读取到某处变更后的值后, 不会再读到比这个值更旧的值。
3. **非顺序一致性**: 多次变更不保证有序。单配置变更并发修改不保证变更有序, 多配置短时间内修改也不保证有序。

配置推送机制基于长轮询(Long Polling)实现:

1. 客户端启动时, 首先从ConfigServer获取全量配置并缓存在本地。
2. 客户端与ConfigServer建立长连接, 定期发送请求询问是否有配置变更。
3. 如果没有变更, 服务端会hold住请求一段时间(通常30-60秒)。
4. 当配置发生变更时, 服务端会立即响应所有相关的长轮询请求。
5. 客户端收到变更通知后, 更新本地缓存并触发配置变更监听器。
6. 同时, 客户端会将配置持久化到本地文件, 作为断网或服务不可用时的备份。

Lion承诺的推送性能指标:

- 推送成功率 > 99.9%
- 99.9%配置变更推送耗时 < 5秒

3. Lion的配置加载顺序和优先级

详细回答: 这是有问题的



Lion的配置加载遵循特定的顺序和优先级，从高到低依次为：

1. **环境变量**：系统环境变量中的配置具有最高优先级。这意味着如果在环境变量中设置了某个配置项，Lion将优先使用该值，而不会从其他来源加载。
2. **本地配置文件**：如果环境变量中没有找到配置，Lion会尝试从本地配置文件中加载。这些文件通常位于应用的特定目录下。
3. **服务端配置**：如果本地配置文件中也没有找到配置，Lion最后会从配置中心服务端获取。

这种加载顺序设计的好处是：

- 提供了灵活的配置覆盖机制，可以通过环境变量快速覆盖配置，而无需修改代码或重新部署
- 支持本地开发和测试，开发人员可以使用本地配置文件而不依赖配置中心
- 即使配置中心服务不可用，应用也能通过本地配置文件正常启动和运行

在实际使用中，这种机制也需要注意一些问题：

- 如果在环境变量或本地文件中设置了配置，那么服务端的配置变更将不会生效
- 监听器只会监听到服务端配置的变更，不会监听到环境变量或本地文件配置的变更

4. Lion如何保障高可用性

详细回答：

Lion通过多层次的机制保障高可用性，承诺服务可用性不低于99.99%：

1. 服务端高可用：

- 多实例部署，支持负载均衡
- 多数据中心部署，支持跨区域容灾
- 服务注册发现机制，自动剔除不健康的节点
- 数据库主从复制，确保数据安全

2. 客户端高可用：

- 本地缓存机制：客户端会缓存配置，即使服务端不可用也能继续工作
- 本地持久化：配置会持久化到本地文件，作为断网或服务不可用时的备份



- 重试机制：网络异常时会自动重试
 - 降级机制：当服务端不可用时，会降级使用本地缓存的配置
3. 数据一致性保障：
- ConsistencyServer同步模块确保多数据中心间的配置最终一致性
 - 配置变更的版本控制，防止配置覆盖
 - 定期全量同步，修复可能的数据不一致

4. 监控与告警：
- 完善的监控指标，包括推送成功率、推送耗时等
 - 配置变更推送失败告警，及时发现问题
 - 服务健康状态监控，自动发现异常节点

Lion的RTO(恢复时间目标)小于30秒，AZ级故障恢复时间小于3分钟，确保在发生故障能快速恢复服务。

5. Lion的使用限制和适用场景

详细回答：

使用限制：

1. 容量限制：
- 单个key容量大小限制：500K
 - 单个appkey下单环境总容量限制：5M
 - 文件配置单个文件大小限制：10M
2. 性能限制：
- 单个配置更改的QPS不应超过1，高频更改只保证最后一次更新能推送到客户端
 - 配置变更后马上读的场景(写与读间隔小于5s)不适用，因为配置变更需要经过服务端推送后客户端才能拿到最新值
3. 一致性限制：
- 不适用于需要强一致性、顺序一致性或因果一致性的场景



- 同一个配置5秒内多次更改只保证最后一次更改值被通知到，不能捕捉每一次的更改

适用场景：

1. **多节点配置分发**：适合需要在多个服务节点间同步配置的场景，如微服务架构中的配置管理
2. **动态调整系统行为**：适合需要在不重启应用的情况下动态调整系统行为的场景，如功能开关、限流阈值调整
3. **应用间配置依赖**：适合应用之间存在配置依赖关系的场景，一处修改多处生效
4. **灰度发布**：适合需要逐步推出新功能或配置的场景，降低风险
5. **集中式配置管理**：适合需要统一管理和审计配置变更的场景，提高配置管理效率和安全性

不适用于：

- 高频配置变更场景(如每秒多次更改同一配置)
- 对配置变更顺序有严格要求的场景
- 配置变更后需要立即(毫秒级)读取最新值的场景

其他可能的面试问题

1. Lion与业界同类产品(如Apollo、Nacos)的对比
2. Lion的客户端实现原理
3. Lion如何实现配置的灰度发布
4. Lion的配置治理功能是什么
5. Lion如何处理客户端初始化失败的情况
6. Lion支持哪些类型的配置
7. Lion的多语言客户端支持情况
8. Lion如何与Spring集成
9. Lion的性能指标如何
10. 如何处理Lion配置获取不到的问题

1. Lion与业界同类产品(如Apollo、Nacos)的对比

详细回答：

Lion作为美团内部的配置中心，与业界主流的Apollo(携程)和Nacos(阿里)相比，有以下特点：

1. 架构对比：

- Lion采用Portal+Console+APIServer+ConfigServer+MetaServer的多层架构，与Apollo的四层架构(Portal+AdminService+ConfigService+Client)类似，都强调了配置管理与配置发布的分离。
- Nacos架构相对简单，合并了配置管理和服务发现功能，没有明显的层次划分。

2. 功能特性：

- Lion深度集成美团技术体系，支持SET、泳道等美团特有概念，更适合美团内部复杂的多环境部署场景。
- Apollo在配置管理粒度上更细，支持应用、集群、命名空间三级管理，权限控制更精细。
- Nacos除了配置中心功能外，还集成了服务发现功能，是一个更全面的微服务基础设施。

3. 一致性模型：

- Lion采用最终一致性模型，通过长轮询机制实现配置推送。
- Apollo也采用最终一致性，同样使用长轮询机制。
- Nacos支持AP和CP两种一致性模型，用户可以根据需求选择。

4. 多语言支持：

- Lion支持Java、C++、NodeJS、Go等多种语言客户端。
- [Apollo官方支持Java和.NET](#)，社区贡献了其他语言客户端。
- Nacos官方支持Java、Go、Python等多种语言。

5. 配置治理：

- Lion提供强大的配置治理能力，支持配置使用数据收集和分析，帮助识别未使用的配置。
- Apollo和Nacos在配置治理方面相对较弱。

总体来说，Lion作为美团内部产品，更加贴合美团的技术体系和业务需求，在配置治理和多环境支持方面有独特优势；而Apollo和Nacos作为开源产品，在社区生态和通用性方面更有优势。

2. Lion的客户端实现原理

详细回答：

Lion客户端的核心实现原理包括以下几个方面：

1. 初始化流程：

- Lion采用懒加载模式，首次调用API时才会触发初始化。
- 初始化时，客户端首先从MetaServer获取元数据信息，包括ConfigServer地址列表。
- 获取成功后，客户端会将元数据持久化到本地文件，作为断网或服务不可用时的备份。
- 如果从服务端获取失败，会尝试从本地文件读取上一次成功获取的元数据。

2. 配置获取机制：

- 客户端启动时，会从ConfigServer获取全量配置并缓存在内存中。
- 同时，配置会持久化到本地文件系统，作为断网或服务不可用时的备份。
- 配置加载遵循优先级：环境变量 > 本地配置文件 > 服务端配置。

3. 长轮询机制：

- 客户端与ConfigServer建立长连接，定期发送请求询问是否有配置变更。
- 请求中包含当前客户端的配置版本号(releaseKey)。
- 如果服务端发现客户端配置版本与最新版本不一致，立即返回最新配置。
- 如果版本一致，服务端会hold住请求一段时间(通常30-60秒)。
- 在hold期间，如果配置发生变更，服务端立即响应；否则超时后返回304状态码。

4. 配置变更通知：

- 当客户端收到配置变更通知后，会更新内存中的配置缓存。
- 同时触发用户注册的配置变更监听器，执行相应的回调逻辑。
- 更新后的配置会持久化到本地文件，作为断网或服务不可用时的备份。

5. 容错机制：



客户端实现了多种容错机制，包括：

- 本地缓存：即使服务端不可用，也能从本地缓存读取配置
- 自动重试：网络异常时自动重试
- 服务端节点选择：当某个ConfigServer节点不可用时，自动切换到其他节点
- 定期全量同步：定期与服务端进行全量同步，修复可能的数据不一致

6. 多语言实现：

- 不同语言的客户端实现了相同的核心逻辑，但在具体实现上有所差异。
- Java客户端是最完整的实现，其他语言客户端基本遵循相同的设计理念。

这种设计使得Lion客户端具有高可用性和高性能，能够在各种复杂环境下稳定运行。

3. Lion如何实现配置的灰度发布

详细回答：

Lion的灰度发布功能是其重要特性之一，允许配置变更先在部分客户端生效，观察一段时间后再全量发布，降低配置变更风险。具体实现如下：

1. 灰度发布流程：

- 在管理端创建灰度发布任务，指定要灰度的配置项和灰度比例(如10%、50%等)。
- 系统会根据当前客户端节点数量，随机选择指定比例的节点作为灰度目标。
- 灰度配置只推送给选中的灰度节点，其他节点仍使用旧配置。
- 观察一段时间后，可以选择全量发布或回滚。

2. 灰度目标选择机制：

- Lion使用客户端的唯一标识(通常是IP+进程ID)作为灰度选择的依据。
- 对标识进行哈希计算，根据哈希值确定是否属于灰度范围。
- 这种方式确保了同一个客户端在灰度期间配置保持稳定，不会因为重新计算而变化。

3. 灰度配置存储：

- 灰度配置与正式配置在服务端分开存储，有独立的版本管理。



- 灰度配置有明确的生命周期，可以设置自动过期时间。

4. 灰度推送机制：

- 当客户端请求配置时，服务端会根据客户端标识判断是否属于灰度目标。
- 如果是灰度目标，返回灰度配置；否则返回正式配置。
- 客户端对灰度配置的处理与正式配置完全相同，无需特殊逻辑。

5. 灰度状态监控：

- Lion提供灰度发布的详细监控，包括灰度节点列表、灰度配置推送状态等。
- 支持查看灰度配置在各节点的生效情况，便于评估灰度效果。

6. 灰度与全量的区别：

- 100%灰度与全量发布有明显区别：100%灰度只针对创建灰度时的节点，不包括之后扩容的新节点。
- 全量发布包括所有节点，包括灰度期间新扩容的节点。

灰度发布功能大大降低了配置变更的风险，特别适用于核心业务配置的变更场景，是Lion配置中心的重要价值点之一。

4. Lion的配置治理功能是什么

详细回答：

Lion的配置治理功能是其独特的优势之一，通过收集和分析配置使用数据，帮助业务方更好地管理配置。具体包括：

1. 配置使用数据收集：

- 客户端会上报配置的使用情况，包括配置获取方式(get、spring、listener、annotation)、获取频率等。
- 上报的数据最终会在服务端按天维度聚合，形成配置使用的全景视图。

2. 配置总量统计：

- 统计各应用的配置总数、配置容量等基础指标。
- 识别配置数量异常增长的应用，及时发现潜在问题。

3. 使用频率分析：

- 分析配置的使用频率，识别高频使用和低频使用的配置。



- 特别关注长期未使用的配置，这些可能是历史遗留的无用配置。

4. 跨服务调用监控：

- 监控配置的跨服务调用情况，了解配置的调用关系。
- 识别不合理的跨服务配置依赖，优化配置架构。

5. 配置变更监控：

- 追踪配置变更历史，包括变更时间、变更人、变更内容等。
- 分析配置变更频率，识别频繁变更的配置，这些可能需要优化设计。

6. 配置推送链路：

- 监控配置从变更到推送到客户端的全链路，包括推送成功率、推送耗时等指标。
- 当推送出现异常时，及时告警并提供详细的问题诊断信息。

7. 配置治理工具：

- 提供配置下线工具，安全地下线无用配置。
- 提供配置迁移工具，便于配置架构优化。

8. 配置治理报告：

- 定期生成配置治理报告，包括配置健康度评分、优化建议等。
- 帮助业务方持续改进配置管理实践。

通过配置治理功能，Lion不仅是一个配置管理工具，更是一个配置运营平台，帮助业务方实现配置的全生命周期管理，提高配置质量和效率。

5. Lion如何处理客户端初始化失败的情况

详细回答：

Lion客户端初始化失败是一个低概率但影响严重的问题，Lion采取了多种措施来处理这种情况：

1. 初始化失败的场景：

- 客户端初始化失败通常发生在以下情况：
 - 客户端访问MetaServer重试三次全部失败

- 本地没有备份文件可读取
- 这种情况多见于：
 - 新机器首次启动(网络可能不稳定且无本地缓存)
 - 大数据任务(频繁启动且可能无文件系统权限)

2. 初始化失败的影响：

- 初始化失败后，客户端处于完全不可用状态，调用任何API都会抛出异常。
- 不同API会抛出不同类型的异常：
 - `Lion.getConfigRepository()`：抛出 `ExceptionInInitializerError`
 - 如果上面的异常被捕获并忽略，继续调用会抛出 `NoClassDefFoundError`
 - `configRepository.get()`：抛出 `LionException`
 - `Lion.get()`：抛出 `LionException`

3. 减小初始化失败影响的措施：

- 对业务用户的建议：
 - 在项目启动过程中，手动调用API触发Lion初始化
 - 确保捕获并正确处理可能的初始化异常
 - 如果初始化失败，及时中止服务启动流程，避免服务以不完整状态运行
- 对依赖Lion的SDK编写者的建议：
 - 考虑Lion初始化失败的情况，提供降级方案
 - 在SDK初始化时检查Lion状态，如果不可用，及时告警并降级
 - 提供配置默认值，在Lion不可用时使用

4. 客户端容错机制：

- 多次重试：初始化时会重试三次，增加成功概率
- 本地缓存：持久化配置到本地文件，作为备份



- 日志记录：详细记录初始化失败的原因，便于问题诊断

5. 服务端支持：

- 提供配置推送链路监控，及时发现客户端异常
- 支持手动重新推送配置，修复可能的数据不一致

通过这些措施，Lion尽可能减小了初始化失败的概率和影响，但仍建议用户在关键业务中做好容错处理。

6. Lion支持哪些类型的配置

详细回答：

Lion支持多种类型的配置，满足不同的业务需求：

1. 按配置形式分类：

- **动态配置**：KV形式的配置，用户可以通过API或管理端直接修改。
 - 单key容量限制：500K
 - 单环境appkey下总容量限制：5M
- **文件配置**：配置以文件形式存在，用户需要在管理端上传配置文件，再分发到客户端。
 - 单个文件大小限制：10M
 - 单环境总容量限制：暂无明确限制

2. 按数据类型分类（动态配置支持的类型）：

- **字符串类型**：最基本的配置类型，存储文本值
- **整数类型**：存储整数，客户端可直接获取为int类型
- **浮点数类型**：存储浮点数值，客户端可直接获取为double类型
- **布尔类型**：存储true/false值，客户端可直接获取为boolean类型
- **JSON类型**：存储JSON格式的配置，客户端可解析为对象
- **List类型**：存储列表数据，格式为JSON数组
- **Map类型**：存储键值对数据，格式为JSON对象



- **Bean类型**：存储Java Bean对象，格式为JSON对象

- **MCC类型**：兼容历史MCC配置格式

3. 按配置范围分类：

- **项目配置**：特定项目(appkey)下的配置
- **分组配置**：项目下特定分组的配置，用于隔离不同业务场景
- **环境配置**：不同环境(dev/test/staging/prod)的配置
- **SET配置**：特定SET下的配置，用于机房隔离
- **泳道配置**：特定泳道下的配置，用于灰度发布
- **公共组件配置**：多个项目共享的配置

4. 特殊配置类型：

- **加密配置**：敏感信息(如密码、密钥)的加密存储
- **引用配置**：引用其他配置的值，实现配置复用
- **表达式配置**：支持简单表达式计算的配置

Lion的多种配置类型支持，使其能够适应各种复杂的配置管理场景，从简单的开关配置到复杂的业务规则配置都能很好地支持。

7. Lion的多语言客户端支持情况

详细回答：

Lion提供了多种语言的客户端实现，满足不同技术栈的需求：

1. Java客户端：

- 最完整的实现，支持所有Lion功能
- 提供多种配置获取方式：API调用、Spring集成、注解方式
- 支持配置监听、配置治理等高级功能
- 版本更新最频繁，建议使用最新版本(如0.8.20+)

2. C++客户端：

- 支持基本的配置获取和监听功能
- 适用于高性能场景，如基础设施服务
- 版本要求：建议使用0.0.6+版本
- 特点：轻量级实现，资源占用少

3. NodeJS客户端：

- 支持基本的配置获取和监听功能
- 适用于前端服务和BFF层
- 版本要求：建议使用3.0.6+版本
- 特点：支持Promise和async/await风格API

4. Go客户端：

- 支持基本的配置获取和监听功能
- 适用于云原生和微服务场景
- 特点：高性能、低资源占用

5. 客户端功能对比：

功能	Java	C++	NodeJS	Go
基本配置获取	✓	✓	✓	✓
配置监听	✓	✓	✓	✓
Spring集成	✓	-	-	-
注解支持	✓	-	-	-
配置治理	✓	部分	部分	部分
灰度发布	✓	✓	✓	✓

SET/泳道支持	✓	✓	仅泳道	✓
文件配置	✓	✓	-	-

6. 客户端选择建议：

- 对于Java应用，优先使用Java客户端，功能最完整
- 对于性能敏感场景，考虑C++或Go客户端
- 对于前端服务，使用NodeJS客户端
- 对于新项目，建议使用最新版本的客户端

7. 客户端版本管理：

- 各语言客户端版本独立演进
- 建议定期升级到最新版本，获取bug修复和新功能
- 特别注意版本兼容性，某些版本可能有重大变更

Lion的多语言客户端支持，使其能够在异构系统环境中提供统一的配置管理体验，是其作为企业级配置中心的重要优势。

8. Lion如何与Spring集成

详细回答：

Lion提供了与Spring框架的集成，使配置管理更加便捷：

1. 基本集成方式：

- 在Spring配置文件中引入Lion命名空间和配置

代码块

```
<beans xmlns="http://www.springframework.org/schema/beans"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xmlns:lion="http://code.dianping.com/schema/lion"
      xsi:schemaLocation="http://www.springframework.org/schema/beans
                        http://www.springframework.org/schema/beans/spring-beans.xsd
```

```
6
7
8
9 <Lion:config />
10 </beans>
11
```

```
http://code.dianping.com/schema/lion
http://code.dianping.com/schema/lion/lion.xsd">
```

2. 配置占位符解析:

- 使用 `@Value` 注解获取Lion配置

代码块

```
1 @Value("${test.placeholder}")
2 private String placeholder;
3
```

- 注意: `@Value` 方式获取的配置是静态的, 首次获取后不会动态更新

3. 动态配置注解:

- 使用 `@LionConfig` 注解实现配置的动态更新

代码块

```
1 @LionConfig("test.placeholder")
2 private String placeholder;
3
```

- 当配置变更时, 注解字段会自动更新为最新值

4. 配置监听:

- 使用 `@LionConfigRepositoryListener` 注解监听配置变更

代码块

Java

```
1 @LionConfigRepositoryListener(appkey = "lion-demo")
2 public class ConfigRepositoryListener implements ConfigListener {
3     @Override
4     public void configChanged(ConfigEvent configEvent) {
5         System.err.println("config event====" + configEvent);
6     }
7 }
8
```

5. Spring Boot集成:

- Lion提供了Spring Boot Starter, 简化集成配置

代码块

XML

```
1 <dependency>
2     <groupId>com.dianping.lion</groupId>
3     <artifactId>lion-spring-boot-starter</artifactId>
4     <version>${lion.version}</version>
5 </dependency>
```

6. 配置优先级:

- Lion配置可以覆盖Spring的属性源
- 可以通过配置调整Lion与Spring配置的优先级

7. 注意事项:

- @LionConfig 注解使用时需要注意:

- 需要先初始化Lion

- 客户端版本<0.10.9时，key需要带appkey前缀
- 服务端删除配置后，注解获取的配置是null而不是默认值(除非配置revertOriginal参数)
- 使用Spring AOP时，需要注意代理模式：

代码块

```
1 <aop:aspectj-autoproxy proxy-target-class="true"/>
2
```

这样可以避免JDK动态代理导致的getter/setter方法找不到问题

通过与Spring的深度集成，Lion使配置管理变得更加简单和强大，特别适合Spring生态系统的应用。

9. Lion的性能指标如何

详细回答：

Lion作为高性能的配置中心，具有优秀的性能指标：

1. 推送性能：

- 推送成功率：>99.9%（配置修改成功后5s内所有客户端成功获取到最新配置的比例）
- 推送耗时：99.9%配置变更推送耗时<5s
- 正常情况下2min内全部推达（系统bug/sdk-bug/灾难场景除外）

2. 读操作性能：

- HTTP读请求QPS：
 - 不同配置：约6500 QPS
 - 相同配置：约7500 QPS
- 客户端读请求QPS：
 - 本地缓存模式：高达150万 QPS
 - 测试场景：500并发-5000QPS，随机300个key获取



- 客户端读取配置几乎不会对应用性能产生影响

3. 写操作性能:

- 写QPS限制:
 - 整体限流: 1000 QPS (所有业务方共用)
 - 单key写限制: 相同配置写约200 QPS
 - 不同配置写: 约2200 QPS
- 大配置写限制:
 - 500K以上的配置限制为1 QPS

4. 服务可用性:

- 整体服务可用性: >99.99%
- RTO (恢复时间目标): <30s
- AZ级故障恢复时间: <3min

5. 客户端性能:

- 内存占用: 轻量级, 即使加载数千个配置也只占用几MB内存
- CPU占用: 极低, 正常使用几乎不会增加CPU负载
- 启动时间影响: 首次加载配置会略微增加应用启动时间, 但影响很小

6. 网络性能:

- 长轮询连接数: 每个客户端实例维持1个长连接
- 网络流量: 极小, 主要是配置变更时的推送流量
- 带宽要求: 很低, 即使在大规模部署环境中也不会成为瓶颈

7. 存储性能:

- 配置读取延迟: <10ms
- 配置写入延迟: <100ms

数据同步延迟: <1s (多数据中心间)

Lion的高性能设计使其能够支撑美团内部数万个应用、数百万配置的规模, 即使在高并发、大规模的环境下也能保持稳定运行。

10. 如何处理Lion配置获取不到的问题

详细回答:

当遇到Lion配置获取不到的问题时, 可以按以下步骤系统排查:

1. 基本配置检查:

- 确认配置是否存在: 在管理端确认对应环境中是否存在该配置
- 确认appkey、group、key是否正确: 注意大小写和空格
- 确认key格式: 如果使用 `Lion.get(key)` 方法, key必须以appkey为前缀

2. 环境配置检查:

- 检查/data/webapps/appenv文件: 确认环境配置是否正确
- 确认env值: 应为dev/test/staging/prod之一
- 确认SET/泳道配置: 如果使用SET或泳道, 确认appenv中配置与管理端一致

3. 客户端版本检查:

- 确认客户端版本: 不同版本的客户端可能有不同的行为
- 检查是否需要升级: 某些问题可能在新版本中已修复
- 多语言客户端特定问题:
 - NodeJS客户端<3.0.0版本从ZK获取配置, 新配置不写ZK会导致获取不到
 - Go客户端string类型无null区分, 未配置的key将返回空串

4. 本地缓存检查:

- 检查本地缓存文件:

^ 代码块

Java

```
1 /data/appdatas/lion/lastWorkingConfig/${appkey}/${key}  
2
```

- 确认缓存文件内容：是否与预期一致
- 如果缓存文件不正确：可以删除后重启应用

5. 日志分析：

- 查看Lion客户端日志：

^ 代码块

```
1 /data/applogs/inf/${appkey}/lion.log  
2 /data/applogs/lion/${appkey}.log  
3
```

- 查找配置加载关键字：

^ 代码块

```
1 ConfigLoaderManager loaded config: ${key} => ${value} from: ${source}  
2
```

- 查找配置变更关键字：

^ 代码块

```
1 >>>>>config change from  
2
```

6. 特殊情况排查：

- 系统变量覆盖：检查是否在代码中设置了同名系统变量

代码块

Java

```
1 System.setProperty("lion-test.xxx", "value")
```

```
2
```

- **多客户端冲突**：检查是否同时使用了Lion和MCC客户端
- **Spring配置冲突**：检查Spring配置文件中是否有同名配置

7. 常见问题解决：

- **监听器失效**：
 - 确认client环境和服务端修改的环境是否一致
 - 检查key是否在系统变量或环境变量中存在
- **项目名称格式问题**：
 - Lion客户端appkey格式通常是 `xxx-yyy-zzz`，以 `-` 分隔
 - 如果appkey格式不符，需要使用特殊方式获取配置
- **配置删除后仍能获取**：
 - 删除本地缓存文件后重启应用

8. 无法解决时的求助：

- 提供详细信息：机器IP、客户端版本、配置key、管理端地址
- 提供日志：客户端日志、应用启动日志
- 提供环境信息：appenv文件内容、应用部署环境

通过这些系统化的排查步骤，大多数Lion配置获取不到的问题都能得到解决。如果问题仍然存在，可以联系Lion团队获取专业支持。